QSAC: Quantum-assisted Secure Audio Communication using Quantum Entanglement, Audio Steganography, and Classical Encryption

Md. Raisul Islam Rifat[a,c], Md. Mizanur Rahman[b,c], Md. Abdul Kader Nayon[a,c], Md Shawmoon Azad[d,c] and M.R.C. Mahdy[d,e,*]

[a]Department of Electrical and Electronic Engineering, Chittagong University of Engineering and Technology, Raozan, Chattogram, 4349, Bangladesh

[b]Department of Computer Science and Engineering, Rajshahi University of Engineering and Technology, Station Rd, Rajshahi, 6204, Bangladesh

[c]Mahdy Research Academy, Dhaka, Bangladesh

[d]Department of Electrical and Computer Engineering, North South University, Bashundhara R/A, Dhaka, 1229, Bangladesh

[e]NSU Center of Quantum Computing, North South University, Bashundhara R/A, Dhaka, 1229, Bangladesh

ARTICLE INFO

Keywords:
Quantum Key Distribution (QKD)
E91 protocol
Entanglement
CHSH inequality
Steganography
ChaCha20-Poly1305
Secure Hash Algorithm (SHA)

ABSTRACT

The emergence of quantum computing poses a significant security threat to the current classical cryptography system since it has the potential to outperform the current classical computer in some specific tasks due to its unique principle of operation. This necessitates finding a method that is resistant to quantum computers to securely transfer information. This research addresses these challenges by proposing a novel method combining quantum key distribution (QKD), specifically the E91 protocol, with the classical encryption-authentication protocol ChaCha20-Poly1305, and concealing information within another message through steganography to securely transfer audio messages. A shared secret key is created between the communicating parties using E91 QKD, which exploits the stellar protection of quantum entanglement against eavesdropping. The shared key is hashed through the SHA–3 hash function to generate a fixed-length, high-entropy symmetric key. The audio message is hidden inside another audio signal through steganography. The steganographic signal is encrypted using ChaCha20-Poly1305 AEAD in order to provide another layer of obfuscation as well as a means to verify integrity. Through rigorous experiments, we validated the robustness of the proposed methodology in both classical and quantum attacks. The processing of secret audio signals of varying duration (00:01:32 to 00:01:36) exhibits consistent encryption results. The encrypted stego audios show high randomness, with an average entropy of 15.9984, an average correlation of 1.4627×10^{-5}, an average UACI of 49.9977%, and an average NSCR of 99.9985%. We demonstrated the safety of the shared key using the CHSH inequality test, where in the presence of an eavesdropper, the CHSH value is much less than $2\sqrt{2}$. In addition, the integrity of the secret audio is also validated through the verification of the authenticator tag generated during the encryption process. Our research offers a novel framework for secure audio transmission, combining classical encryption and authentication methods with QKD to enhance confidentiality, integrity, and resilience against eavesdropping and tampering, ensuring robust end-to-end security.

1. Introduction

The simplest, as well as the most important, form of exchange for human beings is verbal communication. The invention of the Telephone by Alexander Graham Bell in 1876 transformed the verbal communication demography, making the transmission of human voice, which is essentially an audio signal, over long distances possible. In subsequent centuries, technological improvements have broadened the scope of audio transmission. With the rise of the Internet, the amount of information exchanged through audio signals increased rapidly. This increased number of audio transmissions resulted in the development of different encryption techniques and cryptographic algorithms. The most popular among these algorithms are the symmetric AES and the asymmetric RSA encryption schemes. AES

*Corresponding author
mahdy.chowdhury@northsouth.edu (M.R.C. Mahdy)
ORCID(s): 0009-0003-6247-1924 (Md.R.I. Rifat); 0009-0005-9519-8973 (Md.M. Rahman); 0009-0008-1924-8352 (Md.A.K. Nayon); 0009-0005-1318-072X (M.S. Azad); 0000-0003-3737-0315 (M.R.C. Mahdy)

is a block cipher scheme that utilizes multiple matrix operations to encrypt and decrypt data using the same key [1]. Without any knowledge of the cipher key, it is computationally infeasible to reverse the operations performed during encryption. RSA, on the other hand, utilizes prime number factorization to generate public-private key pairs for each user that are used to encrypt and decrypt data [2]. The security of the RSA scheme relies on the astronomical amount of computational power required to obtain the private key from a public key through prime number factorization.

However, with the advent of quantum computers with an increasing number of functional qubits, these classical cryptography and encryption schemes face an imposing threat [3]. In 1996, Lov K. Grover proposed an algorithm to search for an element in an unordered database of size N, using $\sqrt{N}$ quantum queries [4]. Using Grover's algorithm, the number of trials required to brute-force a key of length k reduces from 2^k to $2^{k/2}$. This reduction in the number of brute-force trials effectively reduces the security level of symmetric encryption schemes such as AES [5]. For example, the AES-128 encryption scheme with a pre-quantum security level of 128 reduces to a post-quantum security level of 64, which is much easier to brute-force. In 1994, P.W. Shor presented a quantum algorithm that can quickly find the prime factorization of any positive integer N [6]. As the security of the RSA algorithm relies on the arduousness of prime number factorization to derive private key from public key, it is currently facing an existential threat due to the exponential speed of Shor's algorithm. The time complexity for Shor's algorithm is estimated to be $\mathcal{O}(72(log(N))^3)$, as opposed to $\mathcal{O}(N^3)$ for classical computers.

1.1. Current Challenges

To address these arising challenges, new research is being done in the field of quantum-augmented communication systems. These systems exploit the principles of quantum mechanics to attain secure data transmission. One of the most promising areas of research in the field of quantum communications is Quantum Key Distribution (QKD). QKD protocols work by establishing a secure cryptographic key between two users over an insecure channel [7]. QKD protocols employ properties unique to quantum mechanics, such as the no-cloning theorem [8] and the uncertainty principle [9], which ensure the detection of any eavesdropping attempts and thereby guarantee the security of the key. However, QKD itself does not provide security on its own; rather, it facilitates the establishment and secure exchange of secret keys that are subsequently used by other cryptographic algorithms and encryption techniques to secure the transmitted information. In addition, the applicability of QKD is restricted by a few practical challenges, such as slow key generation, vulnerability to noise, limited operational distance, etc. In resemblance, the research being done in the field of steganography heralds the emergence of an effective information-concealing technique to obscure sensitive information within seemingly harmless transmission. Steganography is the technique of hiding a secret message within another massage in such a manner that it is not discernible that a secret message is embedded [10]. However, the security of any steganographic technique is heavily dependent on the strength of the cryptographic technique employed to encrypt the data [11]. In summary,

- QKD facilitates secure sharing of encryption keys, but secure data transmission requires classical encryption systems.
- In spite of providing improved security, the real-life implementation of QKD is confined due to some practical challenges such as noise vulnerability, slow key generation, etc.
- The security of steganography is dependent on the encryption scheme employed. If the encryption scheme is compromised, data security is at risk.
- Hybrid systems combining classical and quantum cryptography methods defend against different types of attacks, but it complicates the system design and implementation.

1.2. Proposed Solution

The new vulnerabilities in secure data transmission due to quantum computers and the need to oppose them have been the motivation for our research. With the advancement in quantum computing technology, it is imperative to devise innovative cryptographic techniques that can keep data secure in the prospect of an attack from these powerful computers. It is our aim to develop a vigorous solution for secure data communication by combining the strength of quantum communication with hash function, classical encryption and steganography. Hence, the objective of our experimental work is to investigate the viability and credibility of integrating encrypted steganographic techniques with quantum protocols, specifically QKD, to strengthen the security and resilience of audio communication. This

innovative amalgamation of steganography and quantum communication embodies significant furtherance in the field, providing an exceptional and optimistic approach to secure data transmission. This paper introduces a novel approach

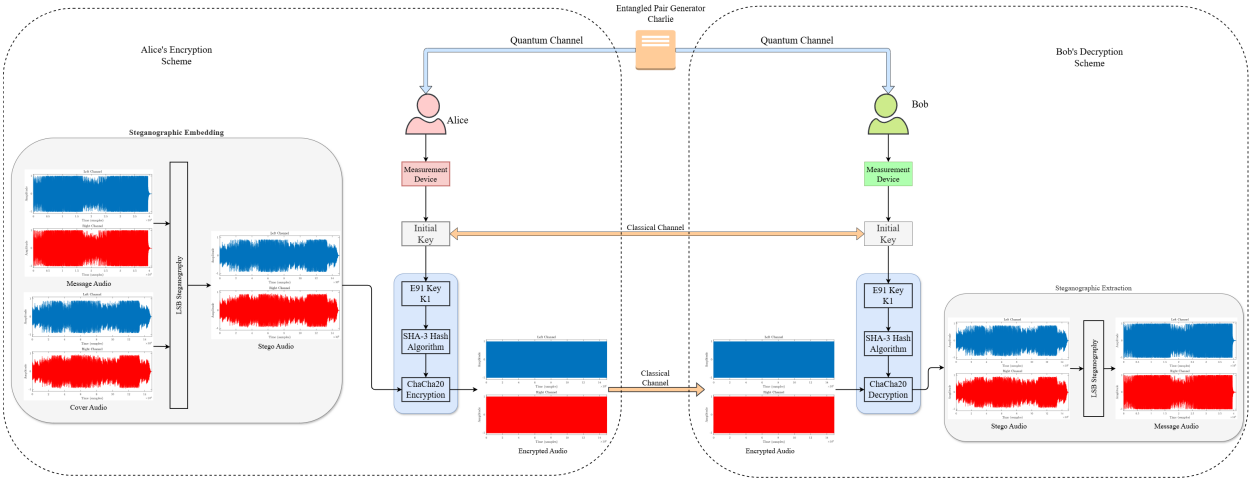


Figure 1: A high-level overview of the proposed hybrid encryption-decryption methodology integrating quantum key distribution, classical authenticated encryption, and steganographic embedding. The architecture uses the E91 protocol to establish a shared Ekert key between Alice and Bob via quantum and classical channels. The derived key undergoes SHA-3 hashing to produce a 256-bit key, which is then used with the ChaCha20-Poly1305 AEAD scheme for encryption. LSB steganography is used to embed and extract secret audio within cover signals, enabling secure and covert communication.

(shown in Figure 1) to reinforce the security of digital audio communication by combining the competency of QKD, classical encryption schemes, and steganography. A high-level outline of our proposed solution is as follows –

- **Secure Key Generation and Distribution:** Utilizing the E91 QKD protocol proposed by Artur K. Ekert in 1991 [12], a shared cryptographic key with enhanced security against quantum threats is generated.
- **Key Derivation:** Employing the Secure Hashing Algorithm-3 (SHA3-256) [13], the QKD-generated key is hashed to produce a fixed-length, high-entropy key for symmetric encryption.
- **Steganography:** Using Least Significant Bit (LSB) substitution steganography, an audio signal is hidden inside another audio signal [14].
- **Symmetric Encryption with Authentication:** Utilizing the ChaCha20-Poly1305 AEAD (Authenticated Encryption with Associated Data) algorithm [15] – which combines the stream cipher scheme, ChaCha20 [16] with the message authentication code, Poly1305 [17] – the steganographic audio is encrypted. The encrypted audio is bundled with a message authentication tag, which is used for authentication and integrity verification.

The incorporation of QKD with classical symmetric encryption addresses various security concerns, providing protection against both classical and quantum threats.

1.3. Novelty of the Proposed Solution

The core novelty of our proposed solution lies in addressing the current limitations in secure communication and bridging them with our respective technical contributions. Each of the subsequent paragraphs defines a specific challenge and describes how our quantum-classical hybrid specifically addresses it.

- One fundamental challenge is that quantum key distribution (QKD) cannot, by itself, secure the real data; it only provides a key, with encryption of data left to classical ciphers. Our solution closes this gap by securely connecting quantum key exchange with a modern authenticated cipher. We use the E91 entanglement-based protocol to establish a shared secret key, then run that key through the SHA3-256 hash function to get a fixed-length, high-entropy symmetric key. The audio payload (now steganographically hidden) is then

encrypted and authenticated by the ChaCha20-Poly1305 AEAD cipher. By feeding the QKD-derived key into ChaCha20-Poly1305, the system gets both quantum-level key protection and strong classical encryption. This strongly integrated solution E91 QKD, SHA-3 hashing, and ChaCha20-Poly1305 leads to a novel hybrid model that solves both classical and post-quantum security requirements. A second inherent challenge is the practical constraints of QKD itself: real-world QKD systems are noise sensitive, have finite range, and generate keys slowly, all of which may slow down secure communication.

- We overcome these disadvantages by taking advantage of the merits of the entanglement-based protocol and effective classical processing. The E91 protocol inherently supports eavesdropping detection via Bell-CHSH tests (aborting keys when deviations are detected), so even noisy environments can be managed. Moreover, and most crucially, we do not utilize raw QKD bits alone; rather, we hash the QKD-generated key with SHA3-256 to produce the ultimate symmetric key. In effect, a small quantum key yield is boosted to full-strength encryption key, enabling ChaCha20-Poly1305 to operate at high speeds. This means the quantum channel supplies secure randomness and the high-speed classical cipher handles data throughput – a strong hybrid approach that overcomes typical QKD performance constraints. Thirdly, while audio steganography can conceal messages, security is immediately contingent on the strength of the underlying encryption. We prevent this by encrypting the stego-audio using a post-quantum secure cipher.

- We hide secret audio in a cover signal using least significant bit (LSB) substitution steganography, then encrypt the stego-signal with ChaCha20-Poly1305 from the SHA3-derived key (using a Poly1305 tag for integrity). Even if the presence of hidden audio is suspected by an attacker, the contents remain protected under the strong AEAD cipher derived from a quantum-secure key. This multi-layered scheme – combining stealthy LSB concealment with authenticated ChaCha20-Poly1305 encryption – is novel, so that the hidden communication is secured both by concealment and by cryptographic security. Finally, hybridizing quantum and classical techniques has a tendency to increase design complexity. We keep this at a minimum by integrating all of the elements into a single end-to-end design.

- Sequentially, our system employs E91-based key exchange for secure key distribution, SHA3-256 hashing for high-entropy key derivation, LSB audio steganography for data hiding, and ChaCha20-Poly1305 encryption for confidentiality and integrity. Each stage is meant to take advantage of some weakness, so the elements complement each other rather than being independent. The result is a consistent hybrid framework: the quantum and classical parts work together, yielding an end-to-end secure audio transmission protocol that is secure both against classical and quantum attacks.

1.4. Paper Organization

Our paper is organized as follows – Section 2 demonstrates the present state of the field through the review of existing research and their development, laying the foundation for our proposed scheme. Section 3 describes the foundational concepts of our proposed schemes. Section 4 describes the proposed methodology, presenting a detailed piecemeal explanation of our proposed architecture. Section 5 delves into a detailed description of our implementation. In section 6, we present the analysis methods as well as the findings. Section 7 explores deeper into an extensive analysis of the results, construing the findings and excerpting key insights. Finally, section 9 concludes the paper by indicating the direction of future work, highlighting the potential application and extension of our research.

2. Related Works

Post-quantum cryptography (PQC), also known as quantum-resistant cryptography, is designed to stave off attacks by both classical and quantum computers. In other words, to design a dependable and future-proof data communication system, a blending of Classical Cryptography and Quantum Cryptography is imperative. One such type of blending could be combining some form of symmetric encryption with quantum key distribution (QKD), as explored in this research.

Classical cryptography, based on the hardness of mathematical problems and computation complexity, has been used to secure information and communication for decades. The classical cryptographic techniques include Symmetric Encryption techniques such as Advanced Encryption System(AES), Data Encryption Standard (DES); Asymmetric Key algorithms such as Rivest-Shamir-Adleman(RSA), Elliptic Curve Cryptography (ECC), and Hash Functions.

These classical cryptography techniques could be the focus of brute force attacks, as they are not capable enough of handling quantum attacks. To address this vulnerability, the algorithms of Post-quantum Cryptography were proposed – which are strong, reliable, and secure against quantum attacks [18].

E91 and BB84 are popular Quantum Key Distribution (QKD) protocols, both founded upon the principles of quantum mechanics. BB84, the first proposed QKD protocol, is founded upon the Heisenberg uncertainty principle and the No-Cloning theorem and makes use of non-orthogonal quantum states to detect eavesdropping [19]. In contrast, the E91 protocol is an entanglement-based protocol and a development of Bell's theorem, utilizing the violation of the CHSH inequality to test for the presence of quantum correlations between distant particles [12]. This enables the users (Alice and Bob) to detect eavesdropping through Bell's inequality violation and ensures security without trusting the entanglement source (Charlie), making E91 more resilient to certain attacks, such as side-channel and source-tampering attacks, and thus more suitable for high-security communications systems. Despite minor problems, E91 is shown to be more dependable and robust than BB84 for uses seeking tighter security guarantees [20].

Hash functions are frequently used in modern cryptography in order to ensure the authentication of the digital communication system. It takes an arbitrary length input and produces a fixed-sized hashed value where any small changes in input result in an alteration of the hashed value. SHA-0, SHA-1, SHA-2 and SHA-3 are the dedicated hash functions of the SHA family [21]. Among them, the SHA-3 family is based on permutation having an extendable output function. SHA3-224, SHA3-256, SHA3-384, and SHA3-512 are known as cryptographic hash functions. SHA-3 provides resistance to collision, preimage and second preimage attacks, which ensures the security of applications such as digital signature generation and verification, key derivation, and pseudorandom bit generation [13]. In the multi-image encryption technique proposed in [22], SHA-3 is employed to strongly associate the encryption keys with the plaintext images, making it more resistant to chosen-plaintext attacks and less likely for key leakage.

Steganography is a common practice of hiding information. The main purpose of steganography is to secure the information so that it is completely undetectable by human senses [23]. There are many types of steganography using different types of cover objects such as text [24], audio [25][26][27], video, images (2D and 3D) [28] and information matrices [29]. Due to their high level of data transmission rate and redundancy, digital audio signals are highly suitable for steganography [30]. Some of the audio stenographic processes include LSB coding, Parity coding, Phase coding, Spread spectrum, and Echo hiding. Among them, LSB is comparatively better as it increases robustness against noise addition and MPEG compression, high capacity and simplicity [31][25]. In the recent literature, steganographic methods have increasingly incorporated compression with chaos-based encryption in an attempt to enhance data hiding ability. For example, [32] proposes a method named ImHALC, which utilizes Local Binary Pattern (LBP)-based texture extraction, compressive sensing, and a novel chaotic map for secure, invisible image steganography. Even though primarily designed for images, the generic concept of combining low-weight compression, adaptive key generation, and chaotic diffusion can be extended at the conceptual level for secure audio steganography. When QKD protocols are combined with steganography, data security is improved. Using the Bernstein-Vazirani algorithm and the BB84 protocol, a secret key is generated which ensures only authorized access to quantum messages. This quantum steganography method increases hidden channel capacity, resists attacks such as intercept-resend, and reduces message detectability by using more Bell states and random information distribution [33].

ChaCha, a variant of Salsa20, is a family of stream ciphers that improves on the previous Salsa20 design. The ChaCha20 is a modification of the 20-round cipher Salsa20/20. It increases the resistance to cryptoanalysis by improving the diffusion per round while maintaining performance. It also achieves better software speed compared to Salsa20 in certain platforms due to its efficient design that allows for SIMD operations [16]. ChaCha20 is a robust alternative to Salsa20, with enhancements that could make it preferable for various cryptographic applications.

The Poly1305 is a specific type of MAC(Message Authentication Code), which is a cryptographic tool used to verify the integrity and authenticity of a message [17]. It computes a 16-byte authenticator tag for variable-length messages using a 32-byte secret key, which includes a 16-byte key derived from some symmetric encryption and a 16-byte additional randomly generated key, which helps in high-speed computations and makes it suitable for various applications [17]. The probability of a successful attack is also very low due to the uniqueness of nonces and the unpredictability of keys. For all these reasons, it is widely used for network protocols and secure communications.

2.1. Existing Literature Gap

The mentioned works offer valuable contributions, but they all have shortcomings as well. QKD, by itself, can not provide any data security. Classical cryptography, along with a hash function as an integrity verifier, provides good data security, but this method is not competent enough to stave off quantum attacks. The consolidation of quantum

and classical cryptography offers a promising avenue for reinforcing security, as the combined strengths of these two approaches can lead to more robust and resilient cryptographic systems. To that end, we introduce a cryptography scheme that combines QKD and classical cryptography, added with the steganography technique. This approach provides a system that is resilient against both classical and quantum attacks while providing robust data security.

3. Preliminaries

In this section, we present the primary concepts employed in our methodology. We start with the quantum key distribution (QKD) protocol proposed by Artur Ekert in 1991, commonly known as the E91 key distribution protocol. Protected by Bell's inequality, E91 protocol provides a secure method of sharing generated key pairs. Then we proceed to the SHA3 family of hashing algorithms, specifically the SHA3-256 function, which can produce a unique 256-bit-long hexstream for an input of any length. Next, we introduce the ChaCha20-Poly1305 AEAD, which combines the strong yet simple ChaCha20 stream cipher with Poly1305 MAC. Finally, we discuss the LSB steganography algorithm, which is essentially a technique of hiding an audio signal in the least significant bits of another larger audio.

3.1. Ekert 91 Protocol

The E91 protocol is a quantum key distribution (QKD) scheme that uses entangled photon pairs and the violation of Bell's inequality to securely distribute cryptographic keys between two parties, typically referred to as Alice and Bob. The E91 protocol, based on the principles of quantum mechanics and entanglement, allows secure key distribution between two parties, Alice and Bob. The steps are as follows:

1. **Creating Entangled Pairs:** Either Alice or Bob or any other third party, Charlie, can initiate the process by generating pairs of strongly entangled quantum particles (Bell states). These states ensure that the measurements on the particles are perfectly correlated. There are four Bell states, which are below:

$$\left|\Phi^{+}\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), |\Phi^{-}\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$

$$\left|\Psi^{+}\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle), |\Psi^{-}\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

   For notions such as $|01\rangle$ and $|11\rangle$, the first digit in the ket notation refers to the first qubit, and the second digit refers to the second qubit.
2. **Sharing the Particles:** After generating the entangled pairs, Charlie sends one particle of each pair to Alice and the other to Bob, or if they produced entangled qubits without the help of a third party, then they will send one particle to another.
3. **Performing Measurements:** Alice and Bob, located far apart, independently measure their received particles using quantum measurement tools. Their devices are configured with randomly chosen azimuthal angles e.g.

$$\varphi_{A_1} = 0, \quad \varphi_{A_2} = \frac{\pi}{2}, \quad \varphi_{A_3} = \frac{\pi}{4} \quad \text{for Alice;}$$

$$\varphi_{B_1} = 0, \quad \varphi_{B_2} = \frac{3\pi}{2}, \quad \varphi_{B_3} = \frac{\pi}{4} \quad \text{for Bob.}$$

   The outcomes (0 or 1) are inherently random due to quantum principles. Figure 2 shows the measurement direction:
4. **Comparing Results:** Alice and Bob will share their measurement bases over a classical channel. They discard results where their measurement angles differed and keep only those whose bases are aligned. This filtered data forms the raw material for their shared key.
5. **Building the Shared key:** For the matching bases, Alice and Bob use their correlated outcomes to construct an identical secret key. This key is a binary sequence that is used to securely encrypt/decrypt messages.
6. **Checking for Eavesdroppers:** The protocol's security depends on the quantum entanglement. If an eavesdropper intercepts the particles, the entanglement is disrupted, altering the expected correlations. By testing a subset of their results, Alice and Bob can detect anomalies. If inconsistencies arise, they abort the key and restart the process.

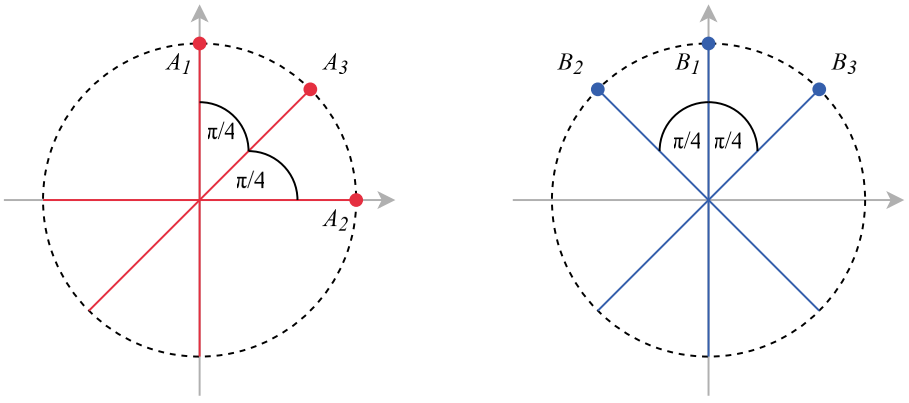


Figure 2: Measurement Direction for Ekert Protocol

Alice and Bob can detect the presence of Eve using two steps. The first step is to check if, after measurement, they find that the measurement results differ despite using the same corresponding bases. They can also detect anomalies using the CHSH inequality test. If the measurement results don't violate CHSH inequality, then they can assume that there was an anomaly. The CHSH inequality quantifies correlations between two spatially separated particles measured in different bases. For an entangled pair, if measurements on the first particle are labeled with settings A and a (yielding outcomes ±1), and those on the second particle use settings B and b (yielding outcomes ±1), the CHSH parameter S is defined as:

$$S = A(B - b) + a(B + b) \tag{1}$$

Classically, the absolute average value is given by:

$$|\langle S \rangle| \leq 2 \quad \text{or} \quad \pm 2.$$

It is bounded by 2, as one of the terms $(B \pm b)$ necessarily cancels out, leaving $S = \pm 2$. Expanding S in terms of A, a, B and b results in:

$$\left|\langle S_1 \rangle\right| = |\langle AB \rangle - \langle Ab \rangle + \langle aB \rangle + \langle ab \rangle| \leq 2.$$

Remarkably, quantum mechanics predicts violations of these classical bounds. Entangled particles, such as those in Bell states, exhibit stronger correlation due to nonlocal quantum interactions. When measured in specific non-commuting bases, the CHSH parameter can reach

$$|\langle S \rangle| = 2\sqrt{2} \approx 2.828$$

In our experiment, we expect to obtain a CHSH value close to $2\sqrt{2}$; otherwise, we can claim that there were anomalies/eavesdroppers in the channel.

3.2. SHA3 – 256 Hash Function

SHA-3 is the latest member [34] of the Secure Hash Algorithm (SHA) family of cryptographic hash functions. Based on the KECCAK algorithm [35], the SHA-3 standard differs fundamentally from its MD-5 [36] like structured predecessors – SHA-1 and SHA-2 [37]. In its essence, the SHA-3 standard is a hash function; that is, for any length of binary data input, there is an output of fixed length. The output is called the *digest* or *hash value*. Depending on the digest length, the four SHA-3 hash functions are called SHA3-224, SHA3-256, SHA3-384, and SHA3-512. The suffixes after the dash indicate the fixed length of the digest; for example, SHA3-256 – which is of interest to our methodology – produces a 256-bit digest. As the digest length is constant irrespective of input length, SHA3 is ideal for integrity and signature verification, as well as password hashing, that is, not storing the password in cleartext format, but rather storing the hash of the password for verification.

SHA-3 works by using the **sponge construction**[38], a design shown in Figure 3 that converts a variable-length input bit string (N) into a fixed-length hash output. The construction operates on an internal state of a fixed width b,

which is divided into a rate (r) and a capacity (c), where $c = b - r$. The process is split into two main phases: absorption and squeezing. The process begins by padding the input message N to ensure that its total length is a multiple of the rate r. The internal state is initialized to a zero-padded bit string of length b. The function then enters the **absorbing phase**. As shown on the left side of the diagram, the padded input message is split into blocks (P) of size r. Each block is XORed ($\oplus$) with the rate portion of the state, after which the entire state is transformed by a permutation function f. In SHA-3, this permutation function used is `Keccak-f[1600]`, which is a series of rounds of bitwise operations (XOR, AND, NOT) that ensure high diffusion and nonlinearity. The capacity c is shielded from this direct input modification, which is crucial for the security of the construction. After all input blocks are absorbed, the function transitions to the **squeezing phase**. An output digest (Z) is initialized to an empty string. The first r bits of the state are appended to Z. If the desired output length has not yet been reached, the permutation f is invoked again on the state to generate a new internal state, and more bits are extracted from the rate portion. This procedure is repeated until Z reaches the required length (e.g., 256 bits for SHA3-256), at which point it may be truncated to form the final hash.

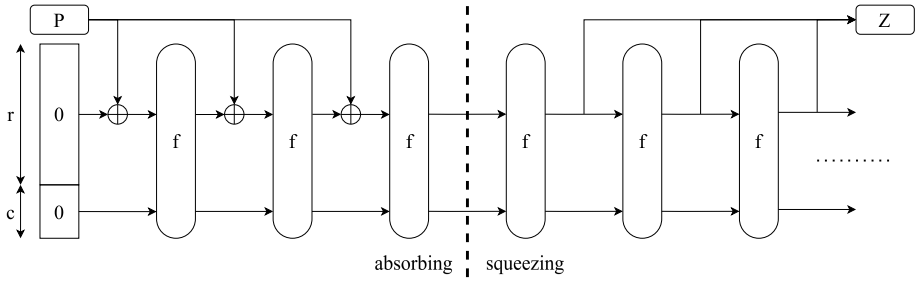


Figure 3: The sponge construction for SHA-3, consisting of an absorbing phase, where input blocks (P) are mixed into the state, and a squeezing phase, where the output hash (Z) is extracted from the state after applying a permutation (f).

For example, for the following binary stream,

$$N = 00100000010001010100000110011000010001001111101010000001100011001111101101011110$$

the digest is $43b3d090d2aebb7790183e58808b71fe890858b5ca3fd308efb2aeba2fde0b71$.

3.3. ChaCha20-Poly1305

ChaCha20-Poly1305 is an authenticated encryption with associated data (AEAD) algorithm that provides a comparatively fast software performance. It is typically faster than AES-GCM (Galois/Counter Mode) in the absence of hardware acceleration [15]. This algorithm combines the ChaCha20 stream cipher [16] and the Poly1305 [17] universal hash family that acts as message authentication code, both of which were developed independently by Daniel J. Bernstein.

A stream cipher is a symmetric key cipher, that is, an encryption and decryption algorithm that combines plaintext data with a pseudorandom keystream [39]. Stream ciphers work by encrypting each plaintext digit one at a time with the corresponding keystream digit, as opposed to block ciphers, where blocks of plaintext with a certain length are encrypted with the keystream. In 2008, Bernstein proposed the ChaCha family of stream ciphers, a successor to the Salsa20 stream cipher [40] proposed by Bernstein in 2005. ChaCha was proposed as an alternative to Salsa20 [41] with slightly better performance. Similar to its predecessor, the initial state of ChaCha is 512-bit long – made up of a 128-bit constant, a 256-bit key, a 64-bit counter and a 64-bit nonce. The 128-bit constant is usually *"expand 32-byte k"*. This 512-bit long input is arranged in a 4×4 matrix where each entry is a 32-bit word. The matrix arrangement is shown below –

$$\begin{bmatrix} \text{Constant[0]} & \text{Constant[1]} & \text{Constant[2]} & \text{Constant[3]} \\ \text{Key[0]} & \text{Key[1]} & \text{Key[2]} & \text{Key[3]} \\ \text{Key[4]} & \text{Key[5]} & \text{Key[6]} & \text{Key[7]} \\ \text{Counter[0]} & \text{Counter[1]} & \text{Nonce[0]} & \text{Nonce[1]} \end{bmatrix} \tag{2}$$

The core operation of ChaCha, and its predecessor Salsa20, is the quarter-round `QR(a,b,c,d)`. ChaCha uses four additions, four XORs and four rotations to update four 32-bit state words – `a,b,c,d`. The update procedure is as follows –

```
a += b; d ^= a; d <<<= 16;
c += d; b ^= c; b <<<= 12;
a += b; d ^= a; d <<<=  8;
c += d; b ^= c; b <<<=  7;
```

If the elements of matrix 2 are indexed from 0 to 15

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

a double round in ChaCha is then defined as –

```
// Odd round
QR(0, 4,  8, 12) // column 1
QR(1, 5,  9, 13) // column 2
QR(2, 6, 10, 14) // column 3
QR(3, 7, 11, 15) // column 4
// Even round
QR(0, 5, 10, 15) // diagonal 1 (main diagonal)
QR(1, 6, 11, 12) // diagonal 2
QR(2, 7,  8, 13) // diagonal 3
QR(3, 4,  9, 14) // diagonal 4
```

ChaCha20 uses 10 iterations of the double round – an overall of 20 rounds. Hence the name ChaCha20.

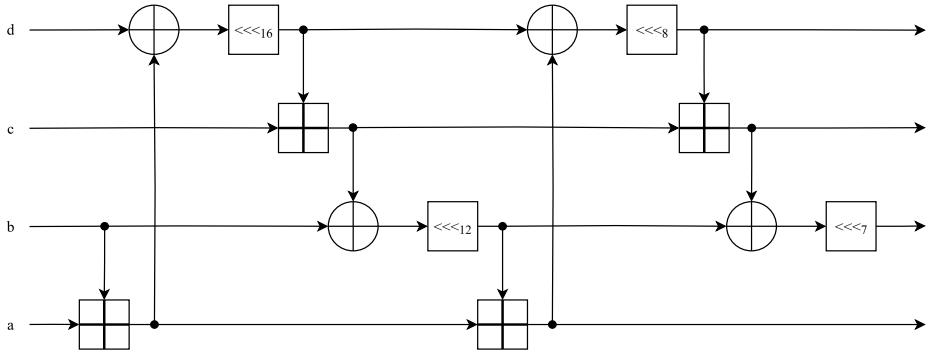


Figure 4: The ChaCha quarter-round function, where four 32-bit state words – `a,b,c,d` – is updated through four additions (⊞), four XORs (⊕) and four rotations (<<<).

Each word is updated twice in ChaCha20's quarter rounds, as opposed to Salsa20, where each word is updated only once. This results in an average of 12.5 output bits to change in each quarter round of ChaCha20, while the Salsa20 quarter round changes 8 output bits. Although the ChaCha quarter round contains the same number of adds, xors, and bit rotates as the Salsa20 quarter round, it is slightly faster because two of the rotate functions are multiples of 8, allowing for a small optimization for x86 and other architectures.

After the 10 iterations, a keystream, K, is generated, which is XORed with the message stream, M, to produce the cipher, C.

$$C = K \oplus M$$

For decryption, the keystream, K, is XORed with the cipher, C, to retrieve the message, M.

$$M = C \oplus K$$

A hash function is a special mathematical function that can convert data of arbitrary size to constant-sized values. The output of a hash function is called hash digest, or simply hashes. A unique property of most hash functions is their

ability to generate unique digests for each unique input. This makes hash functions invaluable for authentication and integrity verification. In 2002, Bernstein designed the Poly1305 universal hash function, which was intended to provide a message authentication code to authenticate a message using a shared secret key [42]. The original implementation of Poly1305 was in 2005 when the Poly1305 hash was combined as a Carter-Wegman authenticator [43] with AES-128 to authenticate encrypted messages.

Poly1305 provides a 128-bit long authenticator tag from a 256-bit long one-time key and a message of arbitrary length. The key has two portions, r and s. It is recommended that the pair (r, s) be unique. The first portion, r, is a 128-bit key produced from a symmetric encryption scheme, such as AES. The latter portion, s, is a 128-bit long integer arranged in a 16-octet little-endian format. It is required that –

- $r[3], r[7], r[11]$ and $r[15]$ have their top four bits clear (to be in $\{0, 1, \ldots, 15\}$).
- $r[4], r[8]$ and $r[12]$ have their bottom four bits clear (to be in $\{0, 4, 8, \ldots, 252\}$).

Each message is required to be accompanied by a 128-bit nonce generated from the accompanying encryption scheme. The process of producing the authenticator tag from the message and the key is briefly outlined here –

- A variable called the "accumulator" is initialized with zeros
- The message is divided into 128-bit long blocks. Each block is read in little-endian format.
- Each block is appended by 1 byte, so that each block is 136-bit long. If any of the block is not 136-bit long (usually the last block), the block is padded with zeros to make it of proper length.
- Each 136-bit long message block is multiplied with a clamped value r, and the multiplication result is added to the accumulator.
- The value stored in the accumulator is reduced using $mod(2^{128})$ and the reduced value is stored in the accumulator.
- When the whole message is converted, the value in the accumulator is added to s.
- From the result, the 128 least significant bits are formatted in little-endian format to produce the tag.

The ChaCha20-Poly1305 AEAD adopted for Internet Engineering Task Force (IETF) [44] and subsequently as the standard for Transport Layer Security (TLS) [45] and OpenBSD Secure Shell (OpenSSH) [46] is slightly different from the original structures proposed by Daniel J. Bernstein. To be precise, the ChaCha20 symmetric encryption is slightly modified. As opposed to a 64-bit Nonce, the modified ChaCha20 has a 96-bit long Nonce. The modified matrix is shown below –

$$\begin{bmatrix} \text{Constant[0]} & \text{Constant[1]} & \text{Constant[2]} & \text{Constant[3]} \\ \text{Key[0]} & \text{Key[1]} & \text{Key[2]} & \text{Key[3]} \\ \text{Key[4]} & \text{Key[5]} & \text{Key[6]} & \text{Key[7]} \\ \text{Counter[0]} & \text{Nonce[0]} & \text{Nonce[1]} & \text{Nonce[2]} \end{bmatrix} \tag{3}$$

The remaining portions of the ChaCha20 scheme remain unaltered. The same is true for the Poly1305 authenticator. A high-level overview of this modified ChaCha20 encryption and Poly1305 authenticator tag generation is outlined below –

- A 256-bit long string is used as key for the ChaCha20 key derivation along with a unique 96-bit long Nonce.
- Using the key and the nonce, the ChaCha20 block function is activated, which produces a 512-bit state.
- The first 256 serialized bits of the 512-bit state are taken as the Poly1305 key – the first 128 bit as r and the next 128 bit as s.
- The 512-bit state is serialized in little-endian format to produce a keystream.
- The keystream is XORed with the message to produce the ciphertext.

- The Poly1305 function is called with the ciphertext and the previously derived key as input to produce the authenticator tag.

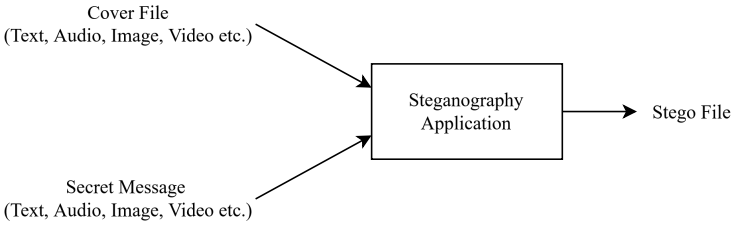
The output from the AEAD is the concatenation of the following –

- A ciphertext which is as long as the plaintext message.
- A 128-bit authenticator tag, generated by the Poly1305 function.

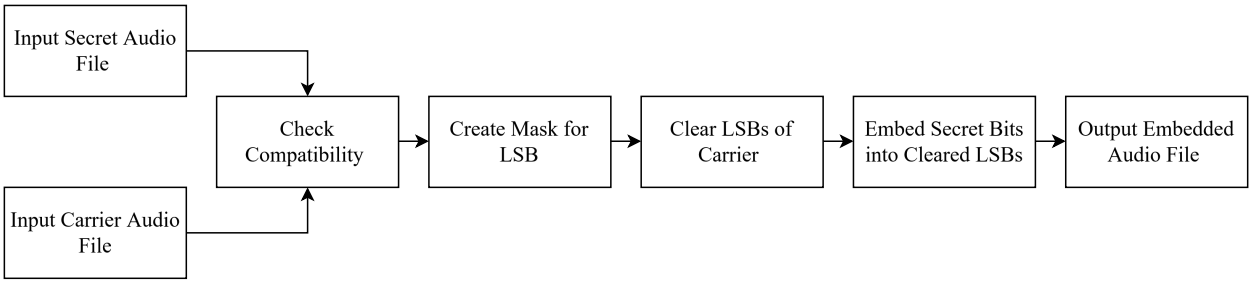
3.4. LSB Steganography

Steganography is the process in which we do not encrypt the secret message; rather, we hide it in another file. Through this process (shown in Figure 5a), we can hide different types of secret data, such as text, image, video, audio, etc., within the cover data, which can also be in any different data types [25][26].

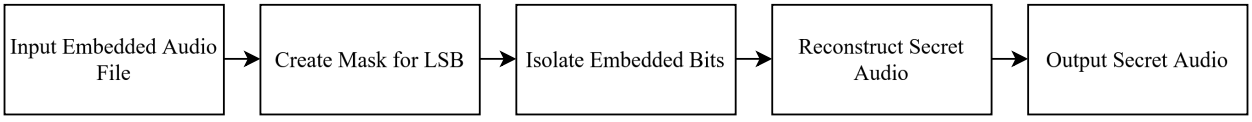
In Audio Steganography, a secret message is hidden in the cover audio, but there is not much change in the cover audio because it uses the psychoacoustical masking phenomenon of the human auditory system (HAS). There are different methods of steganography. Among them, some notables are: LSB Coding, Parity Coding, Phase coding, Spread Spectrum, Echo hiding, etc.[26]. A very popular method among them is the LSB (Least Significant Bit) substitution algorithm, which only replaces the least significant bits of some of the bytes of the cover audio. Due to LSB substitution, there are not many changes in the audio either.



(a) Steganography Application Scenario



(b) Block Diagram of LSB Embedding implementation logic.



(c) Block Diagram for Steganalysis for extracting the embedded data.

Figure 5: Overview of the steganography process, including the application scenario, LSB embedding implementation, and the steganalysis procedure for data extraction.

LSB encoding embeds secret audio by replacing the least significant bits of carrier audio frames with bits from the secret audio. In Figure 5b, first, the carrier and secret audio are checked for compatibility in terms of audio channels and sample width. The secret audio frames are resized to fit the carrier's capacity. A mask is applied to clear the carrier's LSBs, which are then replaced with shifted bits from the secret audio. The result is a modified carrier audio with embedded data, saved as a new file.

Decoding (shown in Figure 5c) isolates the embedded bits by applying a mask on the embedded audio and shifting these bits back to their original positions to reconstruct the secret audio. To improve quality, a low-pass filter is used to reduces noise. The reconstructed audio is clipped to valid ranges and saved as the extracted secret.

4. Methodology

Our proposed system combines the E91 Quantum Key Distribution (QKD) protocol with classical encryption and LSB-based audio steganography to build a secure and hidden audio communication framework. First, entangled photon pairs are shared between Alice and Bob through quantum channels. From the results of their measurements, they generate a shared secret key, known as the Ekert key, using classical communication.

This Ekert key is then passed through a SHA-3 (256-bit) hash function to produce a strong final encryption key. This final key is used in the ChaCha20-Poly1305 algorithm to perform secure and authenticated encryption. Before the encryption step, the secret audio message is hidden inside a cover audio file using Least Significant Bit (LSB) steganography, creating a stego signal.

The encrypted stego signal is then sent to Bob. With the same final key, Bob decrypts the audio and retrieves the hidden secret message using the reverse LSB process. By combining quantum key generation, strong symmetric encryption, and audio steganography, the system ensures high security, data integrity, and covert communication. The complete process is shown in Figure 1.

4.1. Initialization

As outlined by Ekert [12], an entangled pair generator, referred to as Charlie, produces pairs of entangled qubits and shares them with Alice and Bob. The circuit shown in Figure 6 is used to produce the entangled pairs of qubit. The circuit applied a Hadamard (H) gate to put a qubit in superposition and a CNOT ($\oplus$) gate to control the other qubit. Here, qr_0 and qr_1 denote quantum registers, that is, qubits, while cr denotes a classical register.

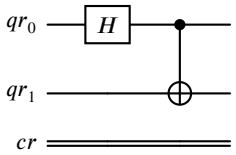


Figure 6: Singlet circuit for creating entanglement qubit pairs.

4.2. Measurement

Alice and Bob independently create a random set of bases of length n. Using this set of random bases, Alice and Bob perform measurements on the qubits they received from Charlie. After the completion of the measurements, both parties share their measurement bases over an unsecured Classical Channel. Upon comparison of these measurement bases, the states that lack a common base are discarded. The result is a secret key called the Final Ekert Key that is shared between Alice and Bob. The measurement process is illustrated in Figure 7.

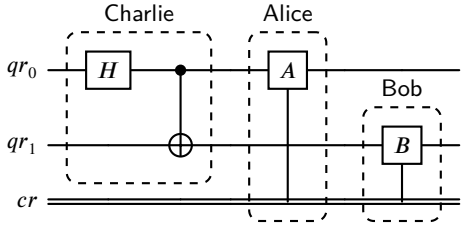


Figure 7: Combined circuit for E91 QKD protocol

4.3. Encryption key generation

After the creation of the final Ekert key, Alice and Bob pass the key through the SHA3-256 hashing algorithm to generate another fixed length (256 bits) and high entropy key. The use of the hashed key instead of the original Final Ekert key provides an additional layer of security by obscuring the contents of the original key. If an eavesdropper gets hold of the hashed key, they will not be able to distill the original key, as the SHA3 hash function provides vital protection against reverse calculation.

For 500 singlet states, we may get a key like the following – 010110001001101001110111101110000110011111100010111000110000001111011101100000010010111011 1. The hash digest of this key is 486340*cb*54*e*65*a*96*edc*02257*b*48*f*3415*a*0*c*374*f*771871*a*4240*ef*8097*cecdeec*2.

Similarly, for 128 singlet states, we may get a key like the following – 00010011101101101110001011 00. The hash digest of this key is 9*e*0*ae*69*c*28*e*4*f*4*e*8*ba*13*e*1*c*496*fb*237284*dfd*4*e*0*b*6540*e*168*b*18*bec*0*cde*9*ee*70*f*.

4.4. Steganographic embedding

Prior encryption, the audio data is embedded in a cover audio using Steganography. The technique of Least Significant Bit (LSB) substitution is used to produce the stego audio, that is, the audio file created from the cover audio that hides the message audio. First, the message and the cover audio are checked for compatibility. Upon success, the cover audio frames are modified through masking, which clears the LSB of each frame. They are subsequently replaced by bits of the message audio.

4.5. Encryption and Authenticator generation

The hashed key generated in 4.3 is used to encrypt the stego audio generated in 4.4. The encryption process is accompanied by the generation of an authenticator tag. The ChaCha20-Poly1305 AEAD is used to perform the encryption. The high-level overview of the process is shown in Figure 8.

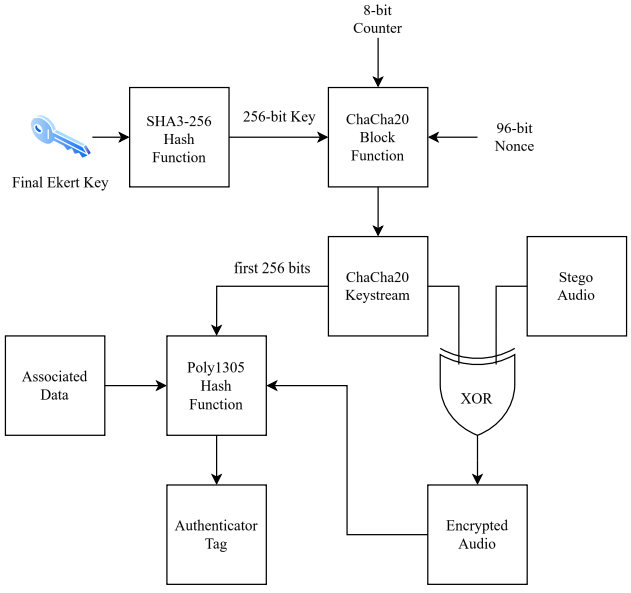


Figure 8: ChaCha20-Poly1305 Encryption and Authenticator tag generation.

The 256-bit key derived from the SHA3-256 hashing algorithm is input along with a 96-bit Nonce and an 8-bit Counter to the ChaCha20 Block function. The function generates a 512-bit state after 20 rounds, which is serialized into a keystream in little-endian format. The first 256-bit of the keystream is input to the Poly1305 hash function along with other associated data. The keystream is XORed with the stego audio to produce the encrypted audio. The encrypted audio is put into the Poly1305 hash function, which produces a 128-bit long authenticator tag.

4.6. Decryption and Authentication

The decryption process is the exact reverse of the encryption process. Similarly to the encryption process, a 256-bit long key is derived from the shared Final Ekert key using the SHA3-256 hash function. The key is used to generate the ChaCha20 keystream, similar to the encryption process. The keystream is used in the Poly1305 hash function along with the encrypted audio received to generate an authenticator tag. The generated tag is compared with the received tag to verify the validity and integrity of the received data. Upon validation, the encrypted audio is XORed with the previously generated keystream to recover the stego audio.

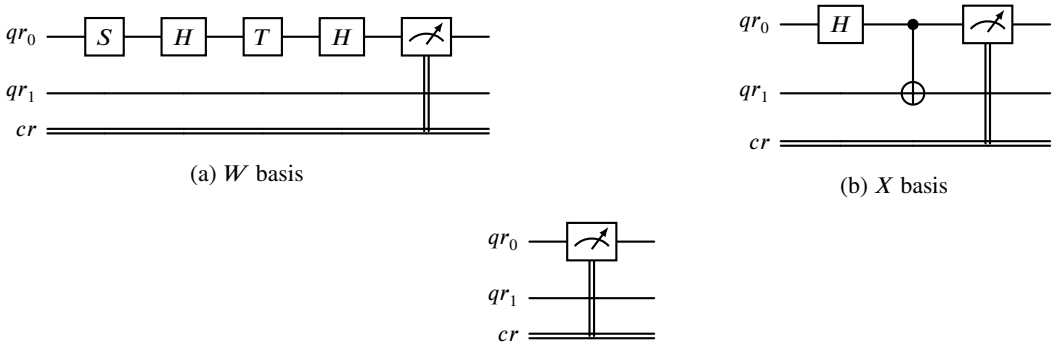
4.7. Steganography extraction

The stego audio recovered in 4.6 is modified to isolate the cover audio and the message audio. As the least significant bits of each frame of the cover audio were replaced by the message audio bits, the LSBs of each frame were extracted in order to reconstruct the binary stream of the message audio. The reconstructed binary stream is modified according to bit-depth to form audio frames. From these frames, the message audio is reconstructed.

In summary, Charlie creates entangled pairs of qubits that are shared with Alice and Bob. They perform measurements on the received qubits, resulting in the creation of the Final Ekert Key. This key is hashed using SHA3-256 to produce a 256-bit high-entropy symmetric encryption key. Alice embeds the secret audio file into a cover audio file through LSB steganography. This embedded audio is encrypted using ChaCha20-Poly1305 AEAD, which uses the symmetric encryption key. The encrypted audio is decrypted by Bob using the same key through ChaCha20-Poly1305 decryption and authenticates the validity of the message. Finally, through LSB extraction, Bob reproduces the secret audio from the decrypted audio.

5. Implementation Details

In the course of our research, we utilized an amalgam of powerful tools and libraries in the pursuit of the effective implementation of our proposed scheme. The Qiskit SDK from IBM [47] acted as one of the foundational columns of our research by enabling us to develop and simulate quantum circuits and thus properly implement the E91 protocol. For our implementation, we specifically used Qiskit v0.39.2. Using Qiskit, we developed the singlet circuit for entangled pair generation 6, the measurement circuits of Alice and Bob and the creation of the Final Ekert Key. The respective measurement circuits are shown in Figures 9 and 10. And we shared the entangled singlet pairs between Alice and Bob in the simulator over an ideal channel; we did not include any photon-loss or channel-noise models in these simulated experiments.



(a) W basis

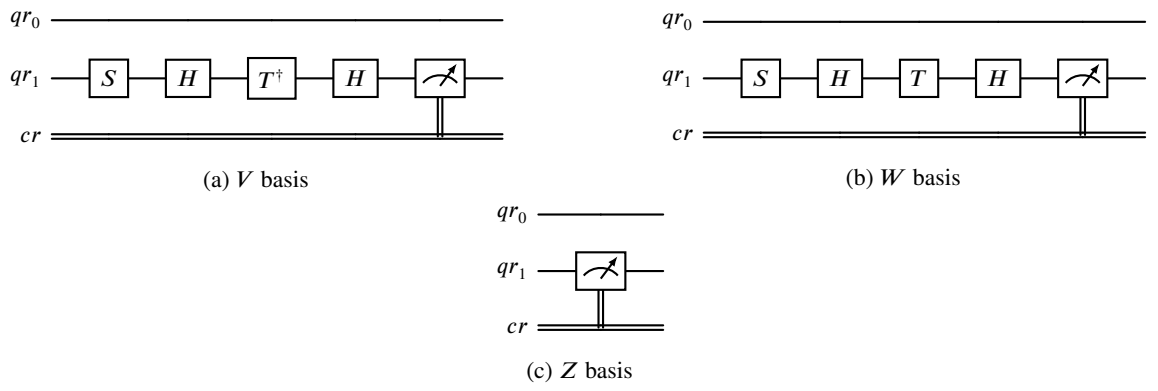
(b) X basis

(c) Z basis

Figure 9: Alice's measurement circuits.

The key was then zero-padded to make it divisible by 8. The zero-padded key was converted into a high-entropy 256-bit long encryption key using PyCryptodome's [48] hash function, specifically the SHA3-256 function, as stated in 4.3.

For our research, we used a single cover audio to hide two secret message audios using LSB steganography following the process detailed in 4.4 using the computational tools and libraries provided by NumPy [49]. For our cover audio, we chose No Place To Go. The original audio was in *.mp3* format. We converted it into *.wav* format using



(a) V basis

(b) W basis

(c) Z basis

Figure 10: Bob's measurement circuits.

FFmpeg [50] and stored it as *cover.wav*. The waveform of the cover audio is shown in Figure 11a. For our secret audios, we chose Alone as secret audio 1 and Stylish Deep Electronic as secret audio 2. Similarly to the cover audio, the secret audios were converted from *.mp3* format to *.wav* format using FFmpeg and saved as *secret1.wav* and *secret2.wav*, respectively. Their respective waveforms are shown in Figures 11b and 11c.

Using the LSB steganography technique, we embedded *secret1.wav* and *secret2.wav* inside *cover.wav* and saved them as *embedded1.wav* and *embedded2.wav*. Their respective waveforms are shown in Figures 12a and 12b. From visual and auditory analysis of the waveforms, we found imperceptible differences among the cover audio and the embedded audios, indicating effective implementation of LSB steganography.

The encryption key was used as the input key to PyCryptodome's ChaCha20-Poly1305 AEAD function in order to generate the cipher keystream as per the procedure stated in 4.5. Using the keystream, the embedded audios, *embedded1.wav* and *embedded2.wav*, were encrypted as ciphertext. From the ciphertext, the authenticator tag was generated. The nonce used in producing the ciphertext, the ciphertext itself and the corresponding tag was stored in *.bin* files, namely, *encrypted1.bin* and *encrypted2.bin*. The *.bin* files were structured in such a way that the 96-bit nonce was followed by the ciphertext, which was, in turn, followed by the 128-bit tag. The waveforms of the encrypted audios are shown in Figure 13a and 13b.

The encrypted file was received by the intended recipient, who decrypted the file and authenticated it by comparing the received tag and the newly computed tag, as illustrated in 4.6. Upon decryption and successful authentication, the audio was stored in *.wav* files, namely, *decrypted1.wav* and *decrypted2.wav*, corresponding to *embedded1.wav* and *embedded2.wav*. The waveforms of these decrypted audios are shown in Figures 14a and 14b. Upon analyzing both visually and aurally, we found that the decrypted audios, *decrypted1.wav* and *decrypted2.wav*, to be indistinguishable from their embedded counterparts, *embedded1.wav* and *embedded2.wav*.

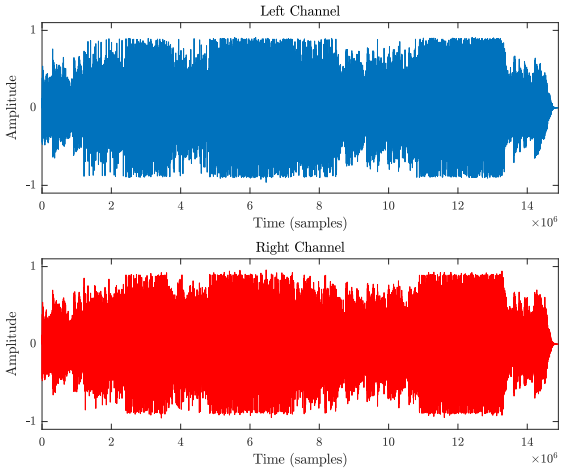
From the decrypted audios, we extracted the secret audio as detailed in 4.7. The extracted audios was stored as *extracted1.wav* and *extracted2.wav*, corresponding to *decrypted1.wav* and *decrypted2.wav*. The waveforms of these audios are shown in Figures 15a and 15b. Visual analysis depicted a slight variation between original secret audios and extracted audios, that is, between 11b and 15a, as well as 11c and 15b. This change was corroborated by auditory analysis. Nevertheless, these slight variations did not make the extracted audio intelligible; instead, it felt like the amplitude of the audio signals was slightly decreased, proving an effective LSB steganographic extraction.

All the audio waveforms shown in this section were generated using MATLAB [51].

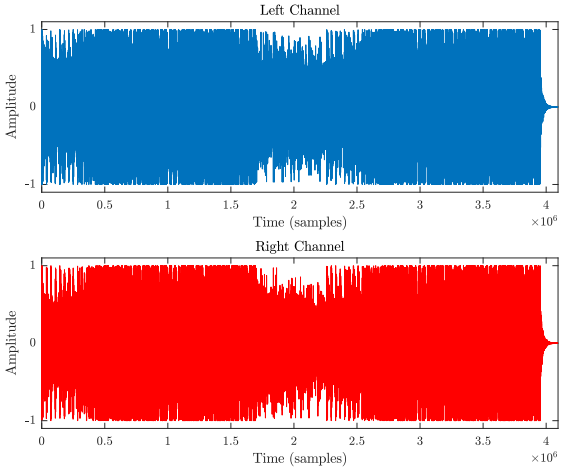
6. Analysis

6.1. Analysis of Encryption Properties

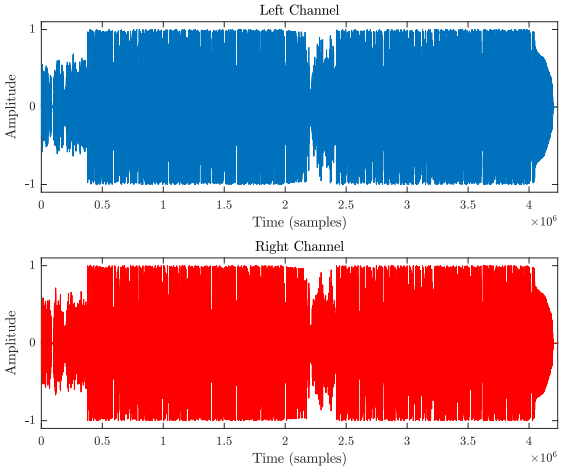
With the help of the tools provided by NumPy, we calculated various properties of the encrypted audio signals obtained in the course of implementing our methodology. These properties range from the entropy of the encrypted signal to sample change rate analysis.



(a) Waveform of Cover audio (*cover.wav*.)



(b) Waveform of Secret audio 1 (*secret1.wav*.)



(c) Waveform of Secret audio 2 (*secret2.wav*).

Figure 11: Waveforms of Cover audio and Secret audios.

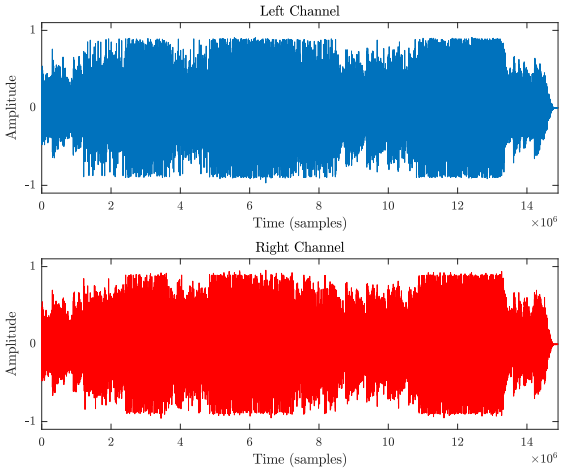
6.1.1. Entropy Analysis

The randomness and unpredictability of a cipher can be measured by the Shanon entropy [52]. In our scenario, the entropy of an encrypted audio signal is an indication of its unpredictability and randomness. The method of calculating entropy is as follows –

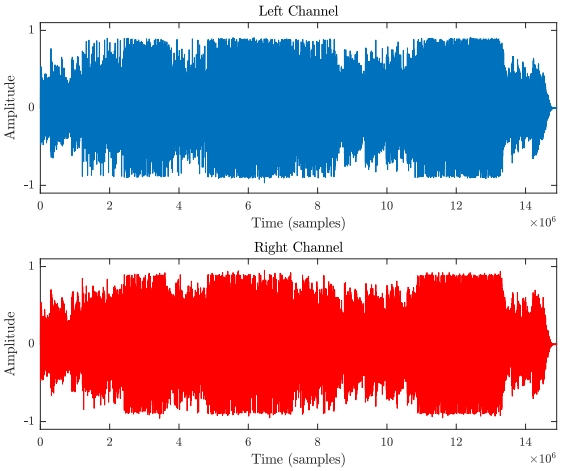
$$H(x) = -\sum_{i=1}^{n} P(x_i) \log_b(P(x_i)) \tag{4}$$

Here, $H(x)$ is the entropy of the signal while $P(x_i)$ is the probability of the ith value of signal x. For a good encryption technique, it is desirable to have the entropy of the encrypted signal to be as close to 16 bits per sample (bps) as possible (for 16-bit audio signals). Table 1 shows the entropy values of the two encrypted signals, as well as those of the embedded signals.

From Table 1, it can be seen that the entropy of the embedded signals is much lower than 16, almost by a margin of 1.6, while that of the encrypted signals is ≈ 16, on average, indicating proper encryption. In addition, the entropy of the decrypted signals being equal to those of the original signals indicates precise decryption.

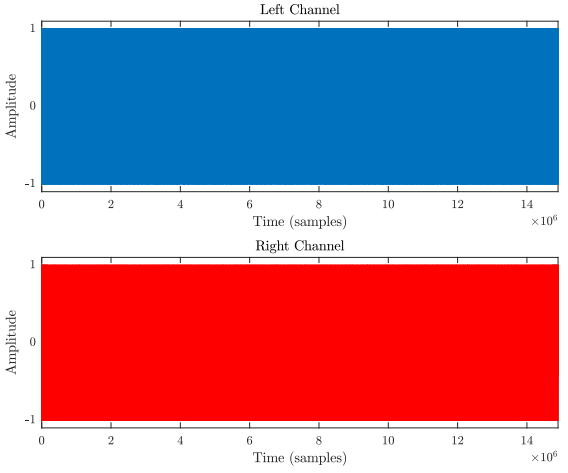


(a) Waveform of Embedded audio 1 (*embedded1.wav*).

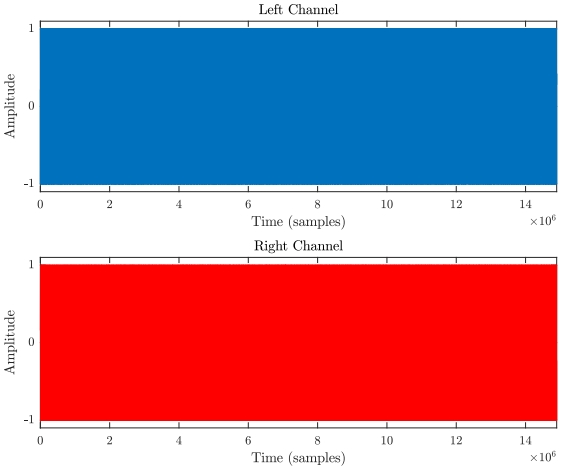


(b) Waveform of Embedded audio 2 (*embedded2.wav*).

Figure 12: Waveforms of Embedded audios produced through LSB Steganography.



(a) Waveform of Encrypted audio 1 (*encrypted1.wav*).



(b) Waveform of Encrypted audio 2 (*encrypted2.wav*).

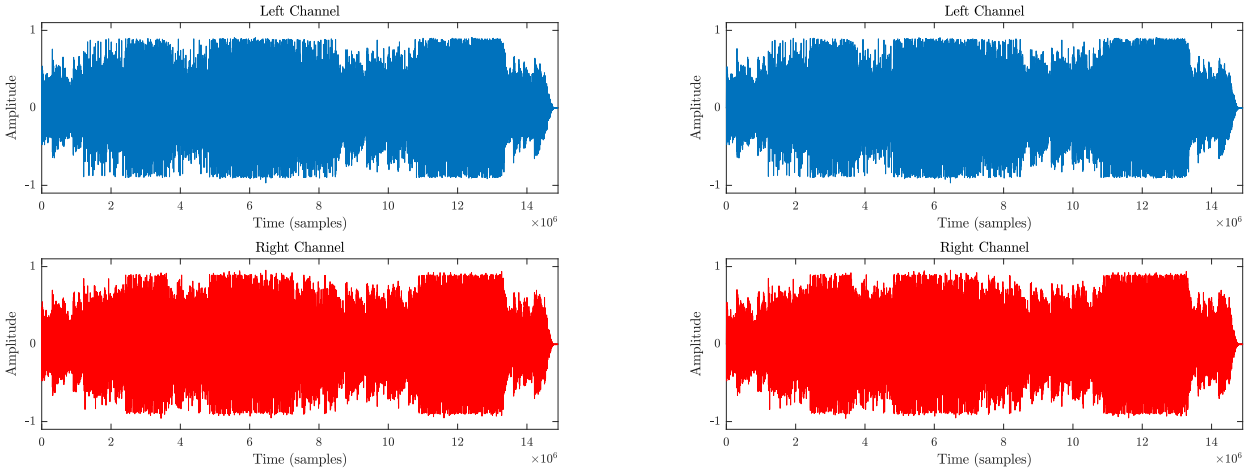
Figure 13: Waveforms of the Encrypted audios produced by encrypting the Embedded audios.

Table 1
Entropy of original embedded audios before encryption, after encryption and after decryption in bits per sample (bps).

Audio File	Original	Encrypted	Decrypted
Audio 1	9.708523634053172 bps	15.998421037422506 bps	9.708523634053172 bps
Audio 2	9.792556077074044 bps	15.998420782458867 bps	9.792556077074044 bps
		avg.: **15.998420909940687** bps	

6.1.2. Correlation Coefficient Analysis

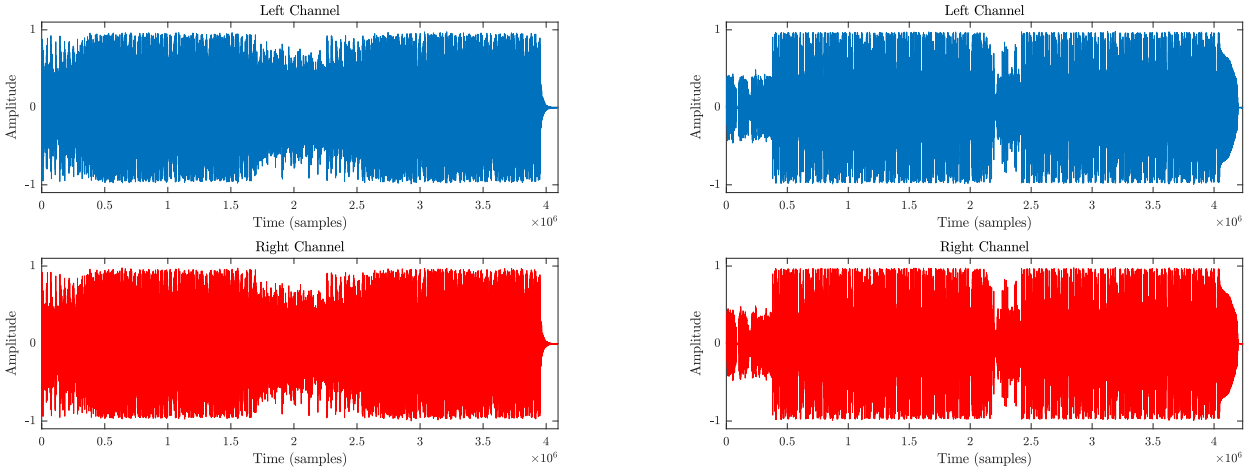
The correlation coefficient provides insight into the relationship between two random variables. For this research, we calculated the correlation coefficients between the adjacent sample of the audio signals before and after encryption.



(a) Waveform of Decrypted audio 1 (*decrypted1.wav*).

(b) Waveform of Decrypted audio 2 (*decrypted2.wav*).

Figure 14: Waveforms of the Decrypted audios produced by decrypting the Encrypted audios.



(a) Waveform of Extracted audio 1 (*extracted1.wav*).

(b) Waveform of Extracted audio 2 (*extracted2.wav*).

Figure 15: Waveforms of the Extracted audios produced through LSB extraction.

The general formula for the correlation coefficient is given by the following –

$$r(x,y) = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2 \sum_{i=1}^{n}(y_i - \overline{y})^2}} \tag{5}$$

Here, x is each sample of both the embedded and encrypted audios, while y is the adjacent sample of both the embedded and the encrypted audios. $\overline{x}$ and $\overline{y}$ are the means of x and y, respectively. The correlation coefficients between the embedded and the encrypted audio signals are shown in Table 2.

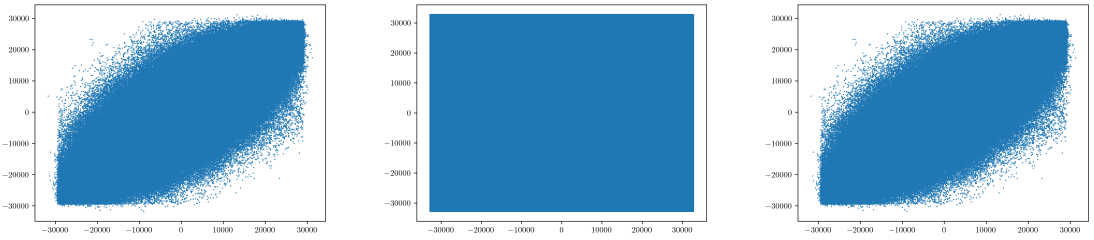
From Table 2, it is evident that both of the encrypted audio signals have a very poor correlation among their corresponding adjacent samples, as both correlation coefficients were well below zero, indicating a high randomness of the encrypted signals. Furthermore, the correlation coefficient for the decrypted signals is the same as those of the original signals, implying proper decryption.

Figures 16 and 17 show the correlation map before encryption, after encryption, and after decryption of Audio 1 and Audio 2 produced through steganography, respectively. The blatant difference between Figure 16a and Figure 16b as well as Figure 17a and Figure 17b is testimony to proper encryption. The similarities between Figure 16a and Figure

Table 2
Correlation between adjacent samples of original embedded audios before encryption, after encryption and after decryption.

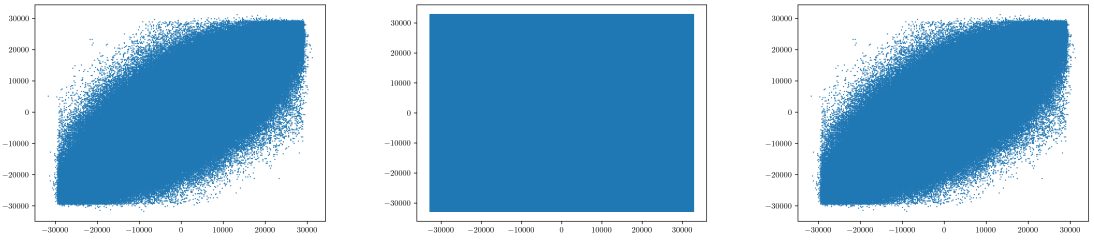
Audio File	Original	Encrypted	Decrypted
Audio 1	0.543752985460514	−0.000019216	0.543752985460514
Audio 2	0.5437499517001024	0.0000484704	0.5437499517001024
		avg.: **0.0000146272**	

16c, as well as Figure 17a and Figure 17c, indicate proper decryption. Furthermore, the accuracy of the steganography scheme is proved by the fact that Figures 16a, 16c, 17a, and 17c are all objectively similar.



(a) Original (b) Encrypted (c) Decrypted

Figure 16: Correlation map of Audio 1: (16a) before encryption, (16b) after encryption and (16c) after decryption.

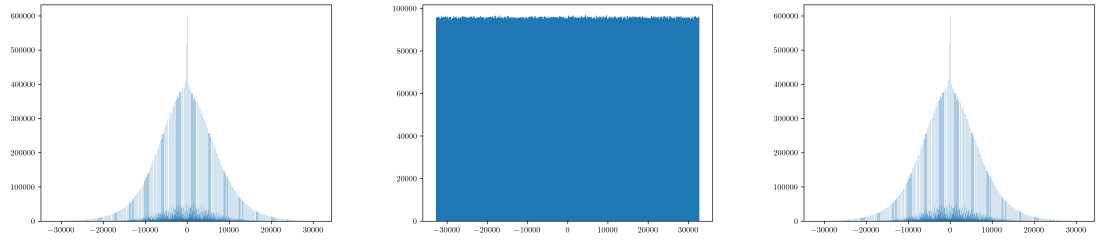


(a) Original (b) Encrypted (c) Decrypted

Figure 17: Correlation map of Audio 2: (17a) before encryption, (17b) after encryption and (17c) after decryption.

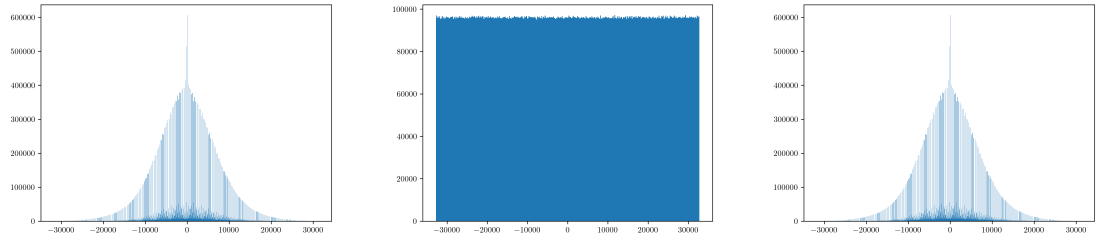
6.1.3. Histogram Analysis

A histogram is a visual representation of how data is distributed. For audio signals, a Histogram depicts the sample frequency with respect to sample amplitude. For a good encryption, the histogram is expected to be uniformly distributed instead of converging or diverging. Figures 18 and 19 show the histogram before encryption, after encryption and after encryption of Audio 1 and Audio 2 produced through steganography, respectively. The histograms depicted in Figures 18 and 19 demonstrate a significant difference between the embedded and encrypted audio signals, confirming the effectiveness of the encryption process. In contrast, the similarity between the histograms of the embedded and decrypted audio signals (Figures 18a and 18c, and Figures 19a and 19c) verifies the accuracy of the decryption process. Moreover, the visual similarity between the histograms of the embedded and decrypted audio signals provides strong evidence for the reliability of the steganography scheme.



(a) Original (b) Encrypted (c) Decrypted

Figure 18: Histogram of Audio 1: (18a) before encryption, (18b) after encryption and (18c) after decryption.

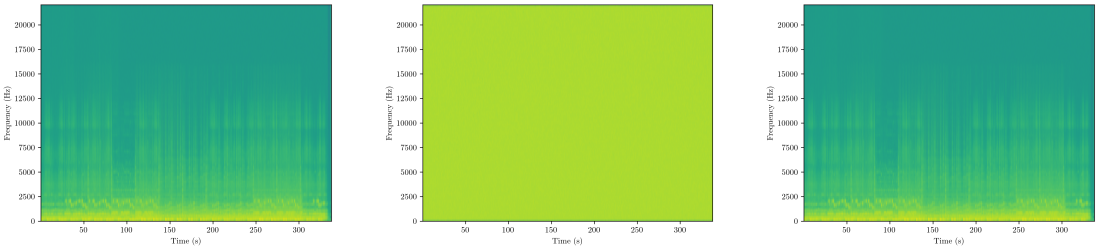


(a) Original (b) Encrypted (c) Decrypted

Figure 19: Histogram of Audio 2: (19a) before encryption, (19b) after encryption and (19c) after decryption.

6.1.4. Spectrogram Analysis

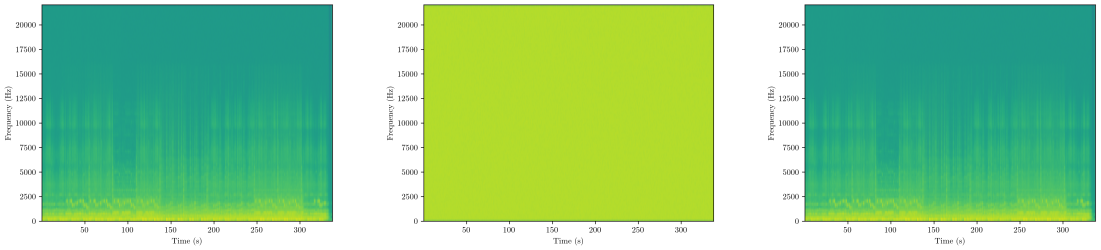
A spectrogram is a visual representation of the variation of the spectrum of frequencies of a signal with respect to time. Figures 20 and 21 show the histogram before encryption, after encryption and after encryption of Audio 1 and Audio 2 produced through steganography, respectively.



(a) Original (b) Encrypted (c) Decrypted

Figure 20: Spectrogram of Audio 1: (20a) before encryption, (20b) after encryption and (20c) after decryption.

A visual inspection of the spectrograms in Figures 20 and 21 reveals a stark contrast between the embedded and encrypted audio signals, indicating successful encryption. Conversely, the spectrograms of the embedded and decrypted audio signals (Figures 20a and 20c, and Figures 21a and 21c) exhibit a high degree of similarity, confirming the correctness of the decryption process. The striking resemblance between these correlation maps also underscores the efficacy of the steganography scheme, demonstrating its ability to conceal and retrieve the embedded audio signals accurately.



(a) Original (b) Encrypted (c) Decrypted

Figure 21: Spectrogram of Audio 2: (21a) before encryption, (21b) after encryption and (21c) after decryption.

6.1.5. Key Sensitivity Analysis

Key Sensitivity is a measure of the response to the slightest change in the key of an encryption system. High Key Sensitivity is an indication of the slightest change in the encryption key, causing huge changes in the encryption and decryption process. To demonstrate Key Sensitivity of the proposed architecture, let us consider two scenarios –

- **Scenario – 1:** The stego-audio is encrypted using K1 key and decrypted using the same key (Figure 22a).
- **Scenario – 2:** The stego-audio is encrypted using K1 key, but decryption is attempted using K2 key, which differs from K1 by a single bit.(Figure 22b)

For instance, let $K1 = \mathbf{0}00010000001000010100011011000011$ and $K2 = \mathbf{1}00010000001000010100011011000011$. The hash output are $\text{SHA3}_{K1} = e57179067dc01f3d3434e4e1447d0368cd716894c71ca9e59da3d7b9a5be6fce$ and $\text{SHA3}_{K2} = a2b702be844ae3adea1fd88c1c6e2452fdde129988fd78f047ebacf5c5d2aa67$. Due to the mismatch in computed hexdigest, the decryption attempt with $K2$ will fail.

6.1.6. Unified Average Changing Intensity (UACI)

Unified Average Changing Intensity is a measure of the average intensity change rate between the original and the modified signal. In this case, UACI measures the average intensity change rate between the embedded audio and the encrypted audio signals. A higher UACI value is preferred, as a higher UACI value indicates a considerable change in the encrypted audio from the original embedded audio. For 16-bit audio signals, UACI is calculated using the following formula –

$$\text{UACI} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{|S_i - D_i|}{65535}\right) \times 100\% \tag{6}$$

Here, S_i is the samples of the embedded signal, D_i is the samples of the encrypted signals, and N is the total number of samples, which is the same in both signals. The calculated values of UACI are shown in Table 3.

Table 3
UACI values of audio signals.

Audio File	UACI
Audio 1	50.003490097286196%
Audio 2	49.991852542937640%
	avg.: **49.997671320111918**%

It can be gleaned with ease from Table 3 that the encrypted audios almost do not contain any patterns from the original embedded audios, as the UACI values were comparatively high.

6.1.7. Number of samples change rate (NSCR)

The number of samples change rate (NSCR) calculates the percentage of sample values that are in contrast between the original embedded audios and the encrypted audios. Higher NSCR values are an indication of a higher number of

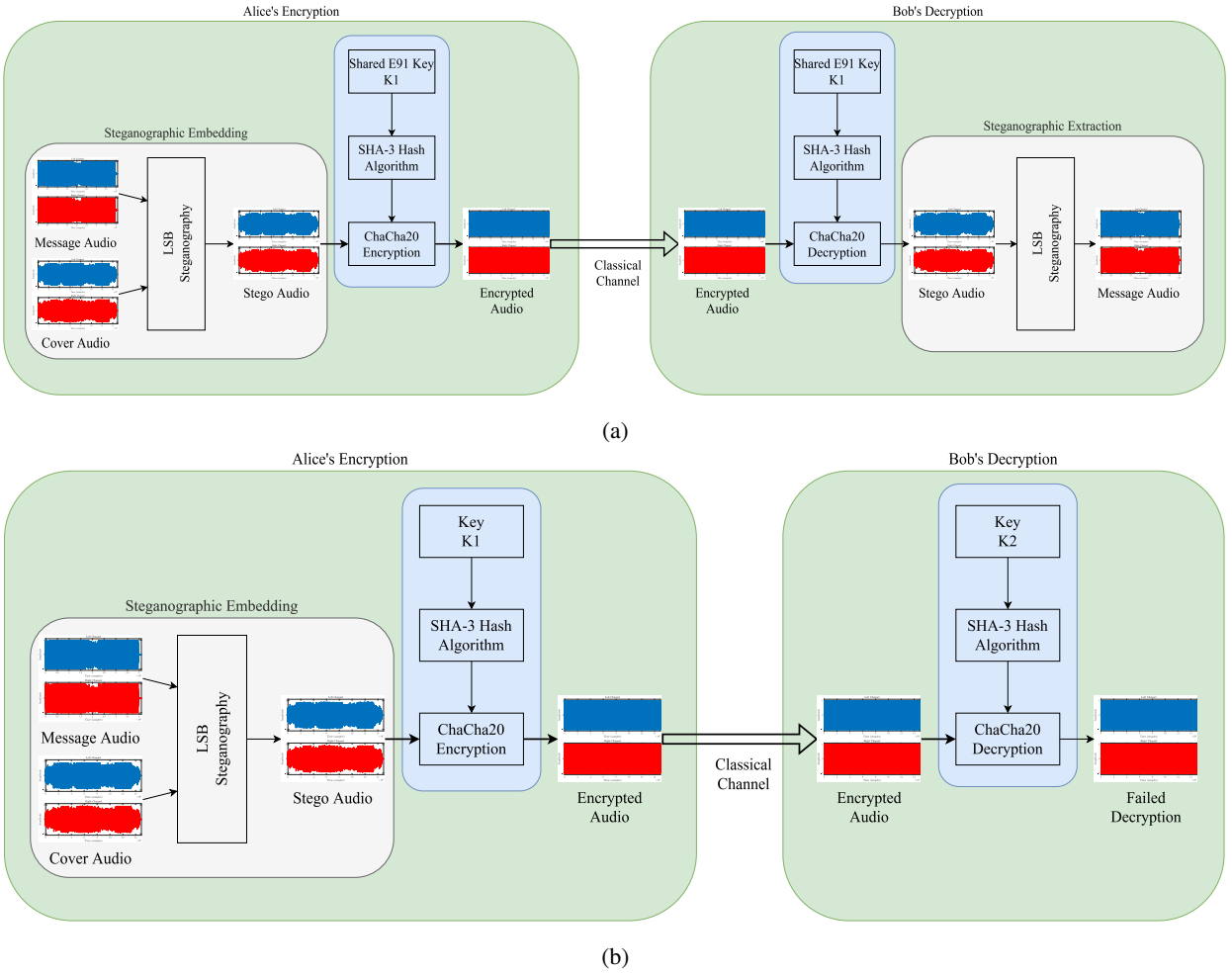


Figure 22: Key Sensitivity Analysis showing two scenarios(a) encryption and decryption using the same key, $K1$, and (b) encryption using key $K1$ with decryption attempted using a different key, $K2$. This illustrates the systems sensitivity to key changes and its robustness against incorrect decryption attempts.

samples being altered in the course of encryption and thus producing a signal substantially different from the original signal. The following formula is utilized to determine the value of NSCR –

$$\text{NSCR} = \left(\frac{D}{N}\right) \times 100\% \tag{7}$$

Here, D is the number of samples that are distinct between the original embedded signal and the encrypted audio signal, and N is the total number of samples, which is the same in both signals. The NSCR values that we determined are shown in Table 4.

Table 4
NSCR values of audio signals.

Audio File	NSCR
Audio 1	99.9985266157303%
Audio 2	99.9984527787054%
	avg.: **99.9984896972179%**

From Table 4, it can be concluded that the encrypted audio signals bear a very small resemblance to their original embedded counterpart, as the NSCR values of both signals were almost close to 100%.

Table 5 presents a definitive comparison of encrypted audio properties of existing works with our proposed scheme. It is evident that our encryption scheme provides improved entropy, correlation, and UACI, all hallmarks of a robust encryption system. Although the NSCR of our scheme is not the highest, it is still close enough to the expected 100%, which indicates high-grade encryption.

Table 5
Comparison of encrypted audio properties for different encryption schemes.

Scheme	Encryption Scheme	Entropy (bps)	Correlation	UACI	NSCR
Ref. [53]	Discrete Modified Henon Map + Modified Lorenz Hyperchaotic system	N/A	0.00971000	33.300000%	99.999500%
Ref. [54]	Lossless Binary Gallos Field Extension-Based Algorithm	15.369518	−0.00260000	33.205726%	99.991687%
Ref. [55]	Self-Adaptive Scrambling + Multi-chaotic Maps + Dynamic DNA Encoding + Cipher Feedback Encryption	N/A	0.00005300	35.548300%	99.994800%
Ref. [56]	Cosine Number Transform (CNT)	15.972500	N/A	33.319200%	99.999213%
Ref. [57]	Cyclic Shift + PWLCM with Elementary Cellular Automata (ECA)	N/A	0.00029518	33.559993%	99.999509%
Ref. [58]	PWLCM + Chebyshev Map + Secure Hash + S-Box (8-bit audio)	7.998300	0.00022000	33.390000%	99.950000%
Ref. [59]	Mordell Elliptic Curves + Permutation + Chaotic Map + S-Box (8-bit audio)	7.943833	−0.00667000	25.585000%	99.583467%
Ref. [60]	Chen Memristor Chaotic System	N/A	0.00095000	32.824200%	99.920275%
This work	**E91 + SHA-3 + Chacha20-Poly1305 + LSB Steganography**	**15.998420**	**0.00001463**	**49.997671%**	**99.998490%**

6.2. Security Analysis

Although our encryption technique provided us with considerably robust encryptions, there exist two different security threats: one in the quantum channel and the other in the classical channel. In the quantum channel, an eavesdropper, Eve, might perform her own measurement on the qubits intended for Alice and Bob. In this process, Eve may gain insight into Alice and Bob's keys. In the classical channel, the encrypted audio suffers from tampering; that is, a malicious party might intercept the encrypted signal and modify it before relaying it to the intended recipient. These scenarios and their solutions are discussed in the subsequent sections.

6.2.1. Quantum Channel Threat Analysis: Resilience against Eavesdropping

The E91 QKD is protected by Bell's inequality. In the presence of an eavesdropper, Eve, the Clauser – Horne – Shimony – Holt (CHSH) correlation [61] will be violated. This serves as a key detection mechanism, alerting Alice and Bob to the presence of an adversary and prompting them to discard the compromised key and take appropriate countermeasures. We considered the following quantum attack vectors in our analysis –

- **Intercept-Resend Attack:** Eve intercepts the traveling qubits, performs a measurement, and then retransmits new qubits to Bob. This process collapses the entanglement between Alice and Bob, significantly degrading the CHSH correlation and making the intrusion detectable.
- **Entanglement-Swapping Attack:** Eve entangles her own qubits with those sent to Alice and Bob using entanglement-swapping techniques. By attempting to remain covert while extracting partial information, Eve can introduce subtle disturbances in the correlation statistics, which nevertheless result in a detectable degradation of the CHSH value.

- **Malicious Source Attack:** In scenarios where Eve controls or replaces the quantum source (Charlie), she may distribute pre-engineered entangled states that mimic legitimate behavior but allow her to retain hidden correlations or bias the measurement outcomes. This compromises the foundational assumption of trusted entanglement and can be revealed through deviation from the expected CHSH threshold.

All of these attack vectors result in degradation of the CHSH correlation. Therefore, to evaluate the robustness of the system against these threats, we present the CHSH correlation values for different key lengths in the presence and absence of an eavesdropper, which were simulated using the Qiskit SDK from IBM. Table 6 displays the CHSH correlation values for different key lengths without eavesdropping, which are close to $2\sqrt{2} \approx 2.828$, indicating that the qubits are maximally entangled and thus confirming the absence of an eavesdropper. In contrast, in Table 7, the CHSH correlation value is below 2, indicating that the qubits are not maximally entangled and thus confirming the presence of an eavesdropper.

Table 6
CHSH correlation values for different key lengths in the absence of an eavesdropper

Iterations	Key Length	Number of Mismatched Bits	CHSH
1	109	0	2.794
2	131	0	2.754
3	215	0	2.636
4	44	0	3.009

Table 7
CHSH correlation values for different key lengths in the presence of an eavesdropper

Iterations	Key Length	Number of Mismatched Bits	CHSH	Eve's Knowledge of Alice's Key	Eve's Knowledge of Bob's Key
1	126	16	1.229	88.1%	96.03%
2	142	12	1.346	93.66%	93.66%
3	243	32	1.37	93.83%	90.53%
4	44	7	1.48	95.45%	84.09%

6.2.2. Classical Channel Threat Analysis: Resilience against Tampering

After storing the secret audio in the cover audio using steganography, we encrypted the embedded audio using the ChaCha20-Poly1305 AEAD. The ChaCha20-Poly1305 produces an encrypted file along with an authenticator tag, which can be used to validate the authenticity of the encrypted file on the receiver end. If a malicious party modifies the encrypted file en route to the receiver, the authentication tag calculated by the receiver – based on the received ciphertext and associated data – will not match the tag sent by the sender. This mismatch indicates that the ciphertext has been altered, causing the verification step to fail and alerting the receiver to the presence of tampering. Since ChaCha20-Poly1305 follows an "encrypt-then-MAC" model, authentication is tightly coupled with the ciphertext and any associated data, ensuring that any unauthorized modification, even a single bit, leads to decryption failure. This authentication mechanism ensures data authenticity (origin integrity) and ciphertext integrity, providing strong resistance to active tampering attacks. Here, we present two different circumstances:

1. The encrypted files, *encrypted1.bin* and *encrypted2.bin*, reach their destination without any tampering by a malicious party. A brief depiction of this scenario is presented in Table 10.
2. The encrypted files, *encrypted1.bin* and *encrypted2.bin*, are intercepted by a malicious party, who performs random byte modification on the files and sends the modified files to their intended destination. A brief depiction of this scenario is presented in Table 11.

In the first scenario, the encrypted files will be successfully decrypted and verified. For the second scenario, we simulated the random byte modifications that the malicious party would have made. As the encrypted file is modified,

the recalculated MAC is different from the one sent alongside the message. This resulted in verification failure and, in turn, decryption failure, alerting the recipient of the actions of the malicious party. Some of these alterations are shown in Tables 8 and 9.

Table 8
Comparison between *encrypted1.bin* and its modified counterpart.

Original File		Modified File	
Address	Hexdata	Address	Hexdata
....	.....	....	.....
000004a0	7f7af670c44b9ffd15b38dc21140951e	000004a0	7f7af670c44b62fd15b38dc21140951e
....	.....	....	.....
000005e0	0ae68fb6740c4347dcfd2b616643d69c	000005e0	0ae68fb6740c4347dc7c2b616643d69c
....	.....	....	.....
00000ae0	ea01590d92bac033b8412dce7cf8e7d7	00000ae0	ea01590d92bac06cb8412dce7cf8e7d7
....	.....	....	.....
....	.....	....	.....

Table 9
Comparison between *encrypted2.bin* and its modified counterpart.

Original File		Modified File	
Address	Hexdata	Address	Hexdata
....	.....	....	.....
00000770	84fa9cb31654d0215b870be20e14f533	00000770	84fa9cb31654d0ad5b870be20e14f533
....	.....	....	.....
00000a60	1d7f2184bd0c5027713aac0e2aa71f64	00000a60	1d7f21846d0c5027713aac0e2aa71f64
....	.....	....	.....
00000ed0	76911249292f57364d80912cec5b5f42	00000ed0	76911249292f57364d17912cec5b5f42
....	.....	....	.....
....	.....	....	.....

7. Findings and Discussion

Our research strived to illustrate the effectiveness of the combination of E91 QKD, ChaCha20-Poly1305 AEAD, and LSB steganography for secure exchange of audio messages. In the preceding sections, we performed different types of analysis to validate our proposed methodology. Figures 12a and 12b provide visual testimony to our LSB embedding scheme, while Figures 13a and 13b demonstrate the effectiveness of our chosen encryption scheme. Figures 14a and 14b illustrate the accuracy of the decryption scheme. Figures 15a and 15b depict the potency of our LSB extraction scheme. We also measured a number of different evaluation metrics to portray the might of our encryption. Table 1 exhibits the Entropy of the encrypted signals, which were 15.9984 on average for both; a further evidence to the strength of the method of our encryption. Table 2 gives tribute to the high randomness of our encrypted signals and their weak relationship with their original unencrypted counterparts. Table 3 presents the UACI values, while Table 4 construes the NSCR values. For both encrypted files, the UACI values were 49.9977%, and the NSCR values were 99.9984%, on average. The obtained values of UACI and NSCR indicate a strong encryption while keeping the distortion low. Furthermore, we performed the CHSH correlation calculation to demonstrate the circumstance of the qubits not being maximally entangled due to the presence of an eavesdropper. In the absence of an eavesdropper, the qubits are maximally entangled, producing a CHSH value around $2\sqrt{2}$, as shown in 6. However, in the presence of an eavesdropper, the qubits will no longer be maximally entangled, resulting in a CHSH value that will deviate substantially from $2\sqrt{2}$. This condition is shown in Table 7. In addition, we performed authenticator tag verification

to demonstrate the effect of a malicious party altering the encrypted files en route to their destination. When the encrypted files arrive intact at their destination without modification, the recipient can successfully verify the integrity of the received file and decrypt it, as shown in Table 10. Nonetheless, if the encrypted files were modified prior to reaching their destination, the integrity verification by the recipient will result in a failure, consequently causing the decryption process to fail. This is shown in Table 11.

Table 10
Scenario 1

	encrypted1.bin	*encrypted2.bin*
Sender End	Nonce: 5e4ef364b086ab3e2d1118cb Tag: 416c6c92659047d2f0660c36f58c1fa5	Nonce: d64f6fe0710ebeb0324e8cc6 Tag: 5977c55fe95a60173fff7476bfad4c29a
Receiver End	Nonce: 5e4ef364b086ab3e2d1118cb Tag: 416c6c92659047d2f0660c36f58c1fa5	Nonce: d64f6fe0710ebeb0324e8cc6 Tag: 5977c55fe95a60173fff7476bfad4c29a
Operation Status	Verification successful! Decryption successful!	Verification successful! Decryption successful!

Table 11
Scenario 2

	encrypted1.bin	*encrypted2.bin*
Sender End	Nonce: 5e4ef364b086ab3e2d1118cb Tag: 416c6c92659047d2f0660c36f58c1fa5	Nonce: d64f6fe0710ebeb0324e8cc6 Tag: 5977c55fe95a60173fff7476bfad4c29a
Receiver End	Nonce:..... Tag:.....	Nonce:..... Tag:.....
Operation Status	Verification failed! Decryption failed!	Verification failed! Decryption failed!

Based on the discussion above, here's how each component in our hybrid scheme contributes a distinct layer of security:

- **E91 Quantum Key Distribution (QKD)** Exploits quantum entanglement and BellCHSH tests to establish an unconditionally secure shared key; any eavesdropping attempt inevitably disturbs the entangled states and is immediately detected.
- **SHA3-256 Hashing** Converts the raw QKD-generated bitstream into a fixed-length, high-entropy symmetric key; provides strong preimage and collision resistance, preventing key-recovery or collision attacks.
- **LSB Audio Steganography** Embeds the secret message within the least significant bits of a cover audio signal; conceals the very existence of hidden data, thwarting detection without knowledge of the embedding pattern.
- **ChaCha20-Poly1305 AEAD** Combines the high-speed ChaCha20 stream cipher with the Poly1305 MAC to deliver authenticated encryption; ensures confidentiality and integrity of the (stego-)audio under the quantum-secure key.

Our research systematically addresses critical limitations in contemporary secure communication systems by integrating quantum and classical cryptographic techniques with steganography in a novel and unified framework. Each design component in our Quantum-assisted Secure Audio Communication (QSAC) system is purposefully crafted to resolve specific challenges highlighted in Section 1.1.

- **QKD facilitates secure sharing of encryption keys, but secure data transmission requires classical encryption systems.**

  Our system explicitly bridges this gap by using the E91 entanglement-based QKD protocol to generate a quantum-secure shared key, which is then processed through the SHA3-256 hash function to derive a

fixed-length, high-entropy symmetric key. This derived key is used within the ChaCha20-Poly1305 AEAD encryption scheme, which ensures confidentiality and authenticity of the audio payload. Thus, the QSAC framework securely transitions from quantum key distribution to classical encryption, ensuring a seamless and secure data transmission pipeline.

- **Despite improved security, real-life QKD is limited by noise vulnerability and slow key generation.**

  The use of E91 QKD in our framework is complemented by the CHSH inequality test to detect eavesdropping and noise anomalies. Moreover, rather than using raw QKD bits for encryption, we amplify the utility of even a small number of entangled key bits by applying SHA3-256, producing a robust encryption key suitable for high-throughput ChaCha20-Poly1305 encryption. This hybrid design efficiently offsets the performance bottlenecks of QKD by leveraging classical post-processing for enhanced throughput without compromising quantum-level security.

- **The security of steganography is dependent on the employed encryption scheme.**

  To resolve this, we employ LSB steganography purely for concealment, while relying on the strength of ChaCha20-Poly1305 for cryptographic protection of the hidden message. This means even if the stego-audio is intercepted and successfully analyzed, the actual payload remains indecipherable without the quantum-secured encryption key. The combination of steganography with a quantum-enhanced AEAD (i.e.,ChaCha20-Poly1305) cipher ensures that both the secrecy and integrity of the embedded message are maintained.

- **Hybrid systems combining classical and quantum cryptography complicate system design and implementation.**

  The QSAC architecture offers a structured and modular solution, where each cryptographic stage (QKD, hashing, steganography, and AEAD encryption) is logically and operationally streamlined. The design integrates each element without unnecessary complexity, ensuring practical implementability. Our proof-of-concept implementation using Qiskit, PyCryptodome, and audio processing libraries confirms that such hybrid systems can be deployed with reasonable computational resources and programming effort.

Overall, our framework offers an effective and practical solution to the growing challenges in secure communication posed by quantum threats.

8. Implications and Significance

Experimental evidence and security testing ensure that our hybrid QSAC system possesses robust resistance to both quantum attacks and classical attacks. By integrating CHSH-based eavesdropping detection with ChaCha20Poly1305 AEAD, the system ensures confidentiality and integrity even when advanced attacks on the communication channel are present. This multilayered defense system ensures that any such tampering effort – either passive quantum eavesdropping or active classical attack – is detected and averted, and the security guarantees of multimedia communication are enhanced over those for purely quantum or purely classical schemes.

Practically, this means that secure audio communication (and, more broadly, multimedia) can be employed with assurance across partially trusted or hybrid networks, such as military beyond-line-of-sight links, financial institution back-channels, or government conferencing systems. The most confidentiality-demanding organizations, such as governments, armies, and banks, are capable of utilizing our scheme in order to safeguard sensitive dialogue from future quantum-powered eavesdroppers without good immunity against the threats of the current time. ChaCha20-Poly1305's low computational footprint and moderate key rates demanded of E91 QKD (amplified by SHA3-256) render this solution a viable alternative for real-time use without unrealistically introducing latency or resource demand.

Besides, the effectiveness of light-weight cryptographic authentication, where undetectable violations of integrity lead to authentication failure, is shown to present a direct route to incorporating quantum-supplemented security into current communication systems. Device manufacturers and communications service providers can increasingly utilize our QSAC solution on top of current conventional VoIP or streaming platforms to render them more immune to both quantum and classical attacks. Finally, by providing evidence of the verification of the system in realistic audio transmission environments of varying lengths and sampling rates, this work paves the way for longerterm research in hybrid quantumclassical protocols, further into the realm of video steganography, multiparty telephone calls, and postquantum key management schemes.

9. Conclusion and Future Work

Our research presents a novel and practical framework for securing digital audio transmission by integrating Quantum Key Distribution, particularly the E91 protocol, with classical cryptographic methods, secure hashing and audio steganography using the Least Significant Bit substitution technique. To enhance both confidentiality and integrity, we integrated the ChaCha20-Poly1305, a modern authenticated encryption scheme known for its performance and security. In our system, entangled photon pairs are generated via the E91 protocol and distributed between Alice and Bob, forming what is known as the Ekert key. This shared quantum key is then processed using the SHA-3 hash function to derive a high-entropy 256-bit symmetric key. This key is used for the ChaCha20-Poly1305 encryption, which encrypts the steganographic audio created by embedding the audio message into another audio, creating a multi-layered security model. Anticipating the challenges posed by future quantum attacks – which could compromise traditional encryption – we tested the robustness of the system through the CHSH inequality test and tampering analysis. These evaluations confirmed the system's resistance to both quantum and classical threats. Furthermore, the audio quality remained consistent after encryption and decryption, supporting the practicality of our approach.

Our future work will focus on (I) experimenting with alternative QKD protocols, such as B92, SARG04, and Coherent One-Way, which may offer improved efficiency or better adaptability to different environments [62]. (II) Enhancing the steganography technique, such as employing phase–or echo–based methods, could improve resistance to detection and data loss. We also plan to (III) extend this framework to support the secure transmission of other data types, such as video. Furthermore, (IV) integrating quantum error correction techniques [63] will help maintain key integrity in noisy channels [64] and enable reliable long-distance key distribution, making the system more scalable and robust for real-world deployment.

References

[1] V. Rijmen, J. Daemen, Advanced encryption standard, Proceedings of federal information processing standards publications, national institute of standards and technology 19 (2001) 22.

[2] R. L. Rivest, A. Shamir, L. Adleman, A method for obtaining digital signatures and public-key cryptosystems, Communications of the ACM 21 (1978) 120–126.

[3] D. J. Bernstein, T. Lange, Post-quantum cryptography, Nature 549 (2017) 188–194.

[4] L. K. Grover, A fast quantum mechanical algorithm for database search, in: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing, pp. 212–219.

[5] J. Daemen, V. Rijmen, The Design of Rijndael, Springer Berlin Heidelberg, 2002.

[6] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, SIAM Journal on Computing 26 (1997) 14841509.

[7] R. Alléaume, C. Branciard, J. Bouda, T. Debuisschert, M. Dianati, N. Gisin, M. Godfrey, P. Grangier, T. Länger, N. Lütkenhaus, et al., Using quantum key distribution for cryptographic purposes: a survey, Theoretical Computer Science 560 (2014) 62–81.

[8] V. Bužek, M. Hillery, Quantum copying: Beyond the no-cloning theorem, Physical Review A 54 (1996) 1844.

[9] D. Sen, The uncertainty relations in quantum mechanics, Current Science (2014) 203–218.

[10] D. Kahn, The history of steganography, Springer Berlin Heidelberg, p. 15.

[11] R. J. Anderson, F. A. Petitcolas, On the limits of steganography, IEEE Journal on selected areas in communications 16 (1998) 474–481.

[12] A. K. Ekert, Quantum cryptography based on bells theorem, Physical Review Letters 67 (1991) 661663.

[13] M. J. Dworkin, Sha-3 standard: Permutation-based hash and extendable-output functions, Federal Information Processing Standards Publication 202 (2015).

[14] N. Cvejic, T. Seppanen, Increasing the capacity of lsb-based audio steganography, in: 2002 IEEE Workshop on Multimedia Signal Processing., IEEE, pp. 336–338.

[15] Y. Nir, A. Langley, ChaCha20 and Poly1305 for IETF Protocols, RFC 7539, 2015.

[16] D. J. Bernstein, et al., Chacha, a variant of salsa20, in: Workshop record of SASC, volume 8, Citeseer, pp. 3–5.

[17] D. J. Bernstein, The poly1305-aes message-authentication code, in: International workshop on fast software encryption, Springer, pp. 32–49.

[18] S. Sharma, K. Ramkumar, A. Kaur, T. Hasija, S. Mittal, B. Singh, Post-quantum cryptography: A solution to the challenges of classical encryption algorithms, Modern Electronics Devices and Communication Systems: Select Proceedings of MEDCOM 2021 (2023) 23–38.

[19] C. H. Bennett, G. Brassard, Quantum cryptography: Public key distribution and coin tossing, Theoretical computer science 560 (2014) 7–11.

[20] L. C. Alvarez, P. C. Caiconte, Comparison and analysis of bb84 and e91 quantum cryptography protocols security strengths, International Journal of Modern Communication Technologies and Research 4 (2016) 265683.

[21] B. Madhuravani, D. Murthy, Cryptographic hash functions: Sha family, Int J Innov Technol Explor Eng 2 (2013) 326–329.

[22] L.-L. Hu, M.-X. Chen, M.-M. Wang, N.-R. Zhou, A multi-image encryption scheme based on block compressive sensing and nonlinear bifurcation diffusion, Chaos, Solitons & Fractals 188 (2024) 115521.

[23] M. M. Amin, M. Salleh, S. Ibrahim, M. R. Katmin, M. Shamsuddin, Information hiding using steganography, in: 4th National Conference of Telecommunication Technology, 2003. NCTT 2003 Proceedings., IEEE, pp. 21–25.

[24] Z.-L. Yang, X.-Q. Guo, Z.-M. Chen, Y.-F. Huang, Y.-J. Zhang, Rnn-stega: Linguistic steganography based on recurrent neural networks, IEEE Transactions on Information Forensics and Security 14 (2018) 1280–1295.

[25] P. Jayaram, H. Ranganatha, H. Anupama, Information hiding using audio steganography–a survey, The International Journal of Multimedia & Its Applications (IJMA) Vol 3 (2011) 86–96.

[26] F. Hemeida, W. Alexan, S. Mamdouh, A comparative study of audio steganography schemes, International Journal of Computing and Digital Systems 10 (2021) 555–562.

[27] F. Djebbar, B. Ayad, K. A. Meraim, H. Hamam, Comparative study of digital audio steganography techniques, EURASIP Journal on Audio, Speech, and Music Processing 2012 (2012) 1–16.

[28] S. Farrag, W. Alexan, Secure 3d data hiding technique based on a mesh traversal algorithm, Multimedia Tools and Applications 79 (2020) 29289–29303.

[29] M. Mashaly, A. El Saied, W. Alexan, A. S. Khalifa, A multiple layer security scheme utilizing information matrices, in: 2019 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA), IEEE, pp. 284–289.

[30] K. U. Singh, A survey on audio steganography approaches, International Journal of Computer Applications 95 (2014).

[31] N. Cvejic, T. Seppanen, Increasing robustness of lsb audio steganography using a novel embedding method, in: International Conference on Information Technology: Coding and Computing, 2004. Proceedings. ITCC 2004., volume 2, IEEE, pp. 533–537.

[32] G. Ye, S. Liu, X. Xiao, X. Hunag, Image hiding algorithm based on local binary pattern and compressive sensing, Mathematics and Computers in Simulation 237 (2025) 316–334.

[33] S. P. Yalla, S. Hemanth, S. T. Kumar, S. Moulali, S. Dileep, Novel text steganography approach using quantum key distribution: Survey paper, NeuroQuantology 20 (2022) 923.

[34] Hash Functions | CSRC | CSRC — csrc.nist.gov, https://csrc.nist.gov/projects/hash-functions, June 22, 2020.

[35] M. P. Guido Bertoni, Joan Daemen, G. van Assche, The keccak sha-3 submission, 2011. [Accessed 26-12-2024].

[36] R. L. Rivest, The MD5 Message-Digest Algorithm, RFC 1321, 1992.

[37] W. Penard, T. Van Werkhoven, On the secure hash algorithm family, Cryptography in context (2008) 1–18.

[38] T. sponge, duplex constructions, Team keccak - guido bertoni, joan daemen, seth hoffert, michaël peeters, gilles van assche and ronny van keer, https://keccak.team/sponge_duplex.html, 2008.

[39] M. J. Robshaw, Stream ciphers, RSA Labratories 25 (1995).

[40] D. J. Bernstein, Salsa20, eSTREAM, ECRYPT Stream Cipher Project, Report 25 (2005) 2005.

[41] D. J. Bernstein, The chacha family of stream ciphers, DJ Bernsteins webpage: http://cr. yp. to/chacha. html (2008).

[42] D. J. Bernstein, Protecting communications against forgery 44 (2008) 535–549.

[43] J. L. Carter, M. N. Wegman, New hash functions and their use in authentication and set equality, Journal of computer and system sciences 22 (1981) 265–277.

[44] Y. Nir, A. Langley, ChaCha20 and Poly1305 for IETF Protocols, RFC 8439, 2018.

[45] A. Langley, W.-T. Chang, N. Mavrogiannopoulos, J. Strombergson, S. Josefsson, ChaCha20-Poly1305 Cipher Suites for Transport Layer Security (TLS), RFC 7905, 2016.

[46] D. Miller, The chacha20-poly1305@ openssh. com authenticated encryption cipher draft-josefsson-ssh-chacha20-poly1305-openssh-00, 2015.

[47] R. Wille, R. V. Meter, Y. Naveh, Ibms qiskit tool chain: Working with and developing for real quantum computers, 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE) (2019) 1234–1240.

[48] Pycryptodome Team, Pycryptodome: a Python library for cryptographic primitives and recipes, https://pypi.org/project/pycryptodome/, 2022.

[49] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (2020) 357–362.

[50] J. Newmarch, J. Newmarch, Ffmpeg/libav, Linux sound programming (2017) 227–234.

[51] MATLAB version 9.0.0.341360 (R2016a), The Mathworks, Inc., Natick, Massachusetts, 2016.

[52] C. E. Shannon, A mathematical theory of communication, The Bell system technical journal 27 (1948) 379–423.

[53] F. Farsana, V. Devi, K. Gopakumar, An audio encryption scheme based on fast walsh hadamard transform and mixed chaotic keystreams, Applied Computing and Informatics 19 (2023) 239–264.

[54] D. Shah, T. Shah, M. M. Hazzazi, M. I. Haider, A. Aljaedi, I. Hussain, An efficient audio encryption scheme based on finite fields, IEEE Access 9 (2021) 144385–144394.

[55] R. I. Abdelfatah, Audio encryption scheme using self-adaptive bit scrambling and two multi chaotic-based dynamic dna computations, IEEE Access 8 (2020) 69894–69907.

[56] J. B. Lima, E. F. da Silva Neto, Audio encryption based on the cosine number transform, Multimedia Tools and Applications 75 (2016) 8403–8418.

[57] P. K. Naskar, S. Bhattacharyya, A. Chaudhuri, An audio encryption based on distinct key blocks along with pwlcm and eca, Nonlinear dynamics 103 (2021) 2019–2042.

[58] A. Adeel, J. Ahmad, H. Larijani, A. Hussain, A novel real-time, lightweight chaotic-encryption scheme for next-generation audio-visual hearing aids, Cognitive Computation 12 (2020) 589–601.

[59] H. Aziz, S. M. M. Gilani, I. Hussain, A. K. Janjua, S. Khurram, A noise-tolerant audio encryption framework designed by the application of s8 symmetric group and chaotic systems, Mathematical Problems in Engineering 2021 (2021) 5554707.

[60] W. Dai, X. Xu, X. Song, G. Li, Audio encryption algorithm based on chen memristor chaotic system, Symmetry 14 (2021) 17.

[61] J. F. Clauser, M. A. Horne, A. Shimony, R. A. Holt, Proposed experiment to test local hidden-variable theories, Phys. Rev. Lett. 23 (1969) 880–884.

[62] A. I. Nurhadi, N. R. Ayambas, Quantum key distribution (qkd) protocols: a survey, in: 2018 4th international conference on wireless and telematics (icwt), ieee, pp. 1–5.

[63] S. J. Devitt, W. J. Munro, K. Nemoto, Quantum error correction for beginners, reports on progress in physics 76 (2013) 076001.

[64] K. Nagata, T. Nakamura, Quantum cryptography, quantum communication, and quantum computer in a noisy environment, international journal of theoretical physics 56 (2017) 2086–2100.